

macOS & APFS Forensics

PARROT OS FIELD MANUAL

Analysing a FileVault2-encrypted APFS image from Debian

Target: macOS Ventura 13 and newer · **Platform:** Parrot OS / Debian

Scope: Dead-box disk analysis · artifact recovery · persistence hunting

TRADITIONAL · ADVANCED · EXOTIC

FOR AUTHORISED FORENSIC EXAMINATION AND EDUCATIONAL USE

§ Contents

ORIENTATION

0 How to use this manual · callout legend · scope

PART I — ACQUISITION & ARRIVAL

1 Acquisition recap — Recovery, csrutil, imaging, SHA-256

2 Evidence intake on the Parrot workstation

PART II — THE PARROT LAB

3 Forensic-mode boot and workstation preparation

4 Write-blocking and safe mounting

5 Case structure, session logging, chain of custody

PART III — OPENING APFS ON LINUX

6 The four access engines

7 Building the toolchain on Parrot

8 Container discovery and layout

9 FileVault2 — keybags, wrapped keys, unlocking

10 Engine capability matrix and known gotchas

PART IV — APFS INTERNALS THAT MATTER

11 Container anatomy — NXSB, checkpoints, transaction IDs

12 Volumes, roles, firmlinks and the System/Data split

13 The catalog B-tree and its record types

14 Time in APFS — four timestamps and what moves them

15 Clones, snapshots, tombstones and copy-on-write

16 Sealed System Volume and cryptexes

PART V — THE TRADITIONAL WORKFLOW

17 The Sleuth Kit run, start to finish

18 Timelines — body files, mactime, super-timelines

19 Carving and unallocated space

20 Plaso, log2timeline and Timesketch

21 Hashing, known-file filtering and YARA sweeps

PART VI — PARROT'S ARSENAL

22 What ships in Parrot, what you must add

23 The toolset, category by category

PART VII — THE ARTIFACT HUNT

24 The unified log — .tracev3 and uuidtext

25 FSEvents — the filesystem's own diary

26 Spotlight metadata stores

27 Quarantine and download provenance

28 Pattern of life — knowledgeC, powerlog, CoreDuet

29 TCC — the permission ledger
30 Notifications, installs and BSM audit logs
31 Browsers, Messages, Mail and user data
32 SQLite deep dive — WAL, freelist, unallocated
33 Plists and NSKeyedArchiver without plutil

PART VIII — THE PERSISTENCE MAP (VENTURA+)

34 launchd — agents and daemons
35 BTM — the modern login-item store
36 Scheduled execution — cron, at, periodic
37 Profiles, MDM and system extensions
38 Plugin and injection surfaces
39 Unix-side persistence
40 The persistence sweep — one-pass checklist

PART IX — THE EXOTIC CHAPTER

41 Checkpoint time-travel
42 Snapshot archaeology
43 XID and inode anomaly detection
44 Orphans, tombstones and dangling extents
45 Free space and the space manager
46 Verifying the seal — detecting a tampered system volume
47 Extended attribute archaeology — com.apple.macl and friends
48 decmpfs and compressed payloads
49 Preboot and Recovery volume analysis
50 The VM volume — swap and sleepimage mining
51 Firmlink confusion and path hiding
52 Slack, sibling links and oversized xattrs
53 Timestamp triangulation — catching a lie
54 Anti-forensics detection
55 Deleted accounts and identity remnants
56 Keychain extraction
57 Mach-O triage without Apple tooling
58 Entropy mapping the container
59 Honest limits — what a disk image cannot tell you

PART X — AUTOMATION AND CORRELATION

60 mac_apt on Linux
61 APOLLO and pattern-of-life correlation
62 Rolling your own over a mounted image

PART XI — REPORTING

63 Findings, manifests and defensibility

APPENDICES

A The full Parrot / Debian install script
B Artifact path cheat sheet (Ventura+)
C Timestamp conversion reference

D SQLite query cookbook
E Command quick-reference card
F Tool capability matrix
G Further reading and sources

0 How to use this manual

This is an anchor document, not a course. It assumes you already know what a filesystem is, why you hash evidence, and roughly what you are looking for. What it gives you is the thing that is genuinely hard to hold in your head: **which command, against which artifact, on which path, under which macOS version** — plus the awkward Linux-side caveats that most macOS-centric guides skip because they assume you are sitting in front of a Mac.

You are not. You are in Parrot OS, which means every convenience Apple builds into `log`, `plutil`, `mdls`, `codesign` and `fs_usage` is unavailable to you. Roughly a third of this manual exists to route around that fact.

Reading order

Parts I–V are the conventional path and are ordered the way an examination actually runs: get the image mounted, understand the filesystem you are looking at, run the classical toolchain, build a timeline. Parts VI–VIII are the reference core — the artifact and persistence maps you will come back to repeatedly. Part IX is the interesting part and assumes everything before it.

If you are mid-case and just need a path or a command, skip to **Appendix B** (artifact paths), **Appendix D** (SQLite queries) or **Appendix E** (command card). The PDF has a navigable bookmark outline for exactly this reason.

Callout legend

> **▲ DEVIL**
>
> A hidden trap, a counter-intuitive behaviour, or an artifact that is easy to walk straight past. These mark the places where an examination quietly goes wrong — where a file looks deleted but isn't, where a timestamp lies, or where evidence lives somewhere no reasonable person would look.

> **■ CAUTION**
>
> Evidence integrity. Something here can modify, destroy or contaminate the image, or produce a finding you will not be able to defend. Read before running.

> **● NOTE**
>
> Context, version differences, or an explanation of why a thing behaves the way it does.

> **▸ TIP**
>
> A shortcut, a better tool, or a way to save an hour.

Command block conventions

Commands are shown at the Parrot prompt. A cyan `$` is an unprivileged command; a magenta `#` means it needs root. Comments are dimmed, and output is shown in plain grey beneath the command.

```
# a comment explaining the intent
└──╼ $ fsapfsinfo -o 0 evidence.dd
fsapfsinfo 20240508
Container information:
        Number of volumes: 5
└──╼ # losetup --read-only --find --show evidence.dd
/dev/loop0
```

Where a placeholder needs substituting it appears in angle brackets — `<IMAGE>`, `<MOUNT>`, `<UUID>`. Throughout the manual the following are assumed:

`<IMAGE>`	your verified raw image, e.g. `/evidence/mbp-001.dd`

<CASE>	your working case directory, e.g. `~/cases/001`
<MNT>	a read-only mountpoint, e.g. `/mnt/apfs`
DATA	the Data volume — where all user content lives on Ventura+
SYS	the System volume — sealed, read-only, signed by Apple

Version tagging

macOS moves paths around constantly. Anything version-sensitive carries a tag:

`13+` Ventura and newer `11-12` Big Sur / Monterey `≤10.15` Catalina and older `ALL` stable across versions

> ■ **CAUTION — SCOPE**
>
> Everything in this manual assumes you have lawful authority to examine the device in question: your own hardware, an engagement with written authorisation, or a lab exercise. The persistence and anti-forensics chapters describe how attackers hide so that you can find them. Applying them in the other direction, or examining a machine you have no right to touch, is not what this document is for.

I

ACQUISITION & ARRIVAL

You have already done this part. It is here for completeness, for the report, and because two details of Apple Silicon imaging change what you can expect to find inside the image once it lands on the Parrot box.

01 Acquisition recap — Recovery, csrutil, imaging, SHA-256
02 Evidence intake on the Parrot workstation

1 Acquisition recap

Kept deliberately short — you have done it. What follows is the shape of the process for the record, plus the parts that materially affect analysis.

1.1 The sequence

```
# --- boot the subject Mac into recoveryOS ---
# Apple Silicon : hold the power button until "Loading startup options"
# Intel/T2      : Cmd-R at chime
# then Utilities > Terminal

csrutil disable           # SIP off — needed for raw block access to disk0
csrutil status            # confirm: "System Integrity Protection status: disabled"

diskutil list             # identify the physical device, NOT the synthesised container

# image the whole physical device to external media
dd if=/dev/rdisk0 of=/Volumes/EVID/mbp-001.dd bs=8m conv=noerror,sync status=progress

# hash source and image, then compare — this is the number that goes in the report
shasum -a 256 /dev/rdisk0
shasum -a 256 /Volumes/EVID/mbp-001.dd
```

● NOTE — WHY /dev/rdisk0

The `r` prefix is the raw character device: unbuffered, and on macOS materially faster than the buffered `/dev/disk0`. Use `disk0`, not `disk1s…` — you want the physical device with its GPT and the entire APFS container, not one synthesised volume. Imaging a single volume throws away the container superblocks, the checkpoint area and every other volume, which between them are the subject of half this manual.

1.2 Two Apple Silicon facts that shape the whole examination

The internal SSD is always hardware-encrypted, and you cannot image it out of the machine. On T2 and Apple Silicon Macs the NVMe is encrypted at rest with a key held in the Secure Enclave. Desolder the chip or pull it into a write-blocker and you get noise. The only way to a usable image is to read the block device from a booted recoveryOS on that specific machine, where the SEP transparently decrypts on the read path. This is why the acquisition is done the way it is, and it is worth a sentence in your report — it explains why the image was not taken with a hardware write-blocker.

FileVault is a second, software layer on top of that. Because the SEP layer is transparent, what lands in your `.dd` is a normal-looking APFS container — but the Data volume inside it is still encrypted by FileVault at the APFS level, with the keys wrapped in the container's keybags. That is the layer you will be unwrapping with the user password in §9. It is also why running `strings` across the raw image returns almost nothing useful until you unlock.

▲ DEVIL — DISABLING SIP TOUCHES THE MACHINE

`csrutil disable` writes to NVRAM and, on Apple Silicon, alters the LocalPolicy on the Preboot volume — which means the act of acquisition left artifacts on the evidence. This is expected and defensible, but you must be able to point at it and say that was me, at that time. Note the exact time you disabled SIP; when you later analyse the Preboot volume (§49) and the unified log, your own footprints will be sitting there. Examiners have mistaken their own LocalPolicy modification for attacker activity more than once.

1.3 What you should also have captured

If the whole physical device was imaged, you have all of these already. Confirm:

- **The complete APFS container** — all volumes, not just Data. Preboot and Recovery matter (§49); the VM volume matters (§50).

- **Any secondary internal device** — on a Fusion setup, the SSD tier and the HDD tier form one logical container and are useless apart.
- **A photograph or note of the machine state** — powered on or off, lid, connected peripherals, whether FileVault was already unlocked.

> **■ CAUTION — WHAT DEAD-BOX COSTS YOU**
>
> A powered-off acquisition permanently forfeits: RAM (and with it running malware, injected code, and the unwrapped volume key), the contents of `/private/var/folders` per-boot temporary directories, established network connections, and anything that lived only in a running process. If the machine was live and you shut it down, say so plainly in the report — that decision is frequently the single largest gap in an examination. §59 covers what this image genuinely cannot answer.

2 Evidence intake on the Parrot workstation

The image has arrived on the analysis box. Before anything touches it, three things happen: verify, immobilise, and record.

2.1 Re-verify the hash

The hash you compare against is the one taken on the subject machine. Transfer corrupts; drives fail silently; USB enclosures lie. Verify on arrival, and record that you did.

```
# Parrot side — must match the shasum -a 256 taken during acquisition
└─$ sha256sum /evidence/mbp-001.dd | tee /evidence/mbp-001.dd.sha256
c3f1...9ab2  /evidence/mbp-001.dd

# if you have the acquisition hash in a file, verify mechanically rather than by eye
└─$ sha256sum -c /evidence/mbp-001.dd.sha256
/evidence/mbp-001.dd: OK
```

▸ TIP — HASH ONCE, IN PARALLEL

A single pass over a 1 TB image costs real time. Compute every digest you will ever want in one read rather than three:

```
└─$ tee < /evidence/mbp-001.dd \
  >(sha256sum > sha256.txt) \
  >(sha1sum   > sha1.txt) \
  >(md5sum    > md5.txt) > /dev/null
```

MD5 and SHA-1 are there only because opposing counsel, older tools and NSRL hash sets still speak them. SHA-256 is the number that matters.

2.2 Immobilise the original

The raw image is now the evidence master. It should never be written to again, and the operating system should be structurally incapable of doing so.

```
# strip write permission from everything, including you
└─# chmod 0444 /evidence/mbp-001.dd
└─# chattr +i /evidence/mbp-001.dd        # immutable — even root cannot write without -i
└─$ lsattr /evidence/mbp-001.dd
----i---------e------- /evidence/mbp-001.dd
```

● NOTE

`chattr +i` needs an ext2/3/4, XFS or btrfs filesystem underneath. If your evidence store is exFAT or NTFS — common on shared external drives — the flag will fail and you are relying on permissions and discipline alone. Consider that a reason to keep evidence on a native Linux filesystem.

2.3 Consider a forensic container format

Raw `.dd` is universally readable and that is its whole virtue. But an E01/EWF container carries its own hashes, compresses well (an APFS image of a half-empty SSD compresses hard), and records case metadata inside the evidence file itself. Every tool in this manual can read it through `ewfmount`.

```
# convert raw to E01 with embedded metadata and verification
└─╼ $ ewfacquire -t /evidence/mbp-001 -f encase6 -c deflate:best -S 2G \
    -C "CASE-001" -D "MacBook Pro 14 M2 internal NVMe" \
    -E "1" -e "J. Examiner" -N "Dead-box image, FileVault2 active" \
    -d sha256 /evidence/mbp-001.dd

# verify the container against its own stored digests
└─╼ $ ewfverify /evidence/mbp-001.E01
Verification completed... MD5 hash stored in file: ... (MATCH)
```

To use it, expose it as a flat file and treat that path as the image everywhere else in this manual:

```
└─╼ # mkdir -p /mnt/ewf && ewfmount /evidence/mbp-001.E01 /mnt/ewf
└─╼ $ ls -l /mnt/ewf/
-r--r--r-- 1 root root 494384795648 Jan  1  1970 ewf1
# /mnt/ewf/ewf1 is now a read-only raw image
```

2.4 Work from a copy, or don't work at all

Analysis happens against a working copy, or against the master through a read-only loop device and a copy-on-write overlay (§4). Never directly. Disk is cheap; a contaminated master is a dead case.

`/evidence/`	read-only master image, immutable, hashed. Mounted `ro` if on its own volume.
`~/cases/001/work/`	working copy or COW overlay — where mistakes are allowed to happen
`~/cases/001/out/`	everything your tools produce
`~/cases/001/notes/`	contemporaneous notes and session transcripts

II

THE PARROT LAB

Parrot ships with a forensic posture that most distributions do not: a boot mode that refuses to touch attached storage, and a toolset that is already two-thirds of what you need. This part sets the machine up so that the filesystem cannot betray you, and so that everything you do is recorded.

03 Forensic-mode boot and workstation preparation
04 Write-blocking and safe mounting
05 Case structure, session logging, chain of custody

3 Forensic-mode boot and workstation preparation

3.1 Parrot's forensic mode

Booting the Parrot live ISO and choosing **Forensics Mode** from the boot menu changes three behaviours that matter:

- **No automount.** Attached block devices are detected but never mounted. On a normal desktop boot, plugging in the evidence drive can trigger an automount that updates access times, replays a dirty journal, or writes a `.Trash-1000` directory.
- **No swap.** Existing swap partitions on attached disks are not activated. This is the big one: a stock Linux install will happily `swapon` a partition it finds on the evidence drive and overwrite it.
- **No persistent state written to attached media.** The live session stays in RAM.

> ■ **CAUTION — INSTALLED PARROT IS NOT FORENSIC MODE**
>
> Forensics Mode is a property of the live boot. If you are working from an installed Parrot system — which most people are, because analysis needs persistence and disk space — you do **not** get any of the above. You must disable automount yourself. Do this once, permanently, on any machine that will see evidence.

3.2 Hardening an installed Parrot for evidence work

```
# --- kill automounting at the source ---
└─# systemctl stop udisks2 && systemctl disable udisks2 && systemctl mask udisks2

# GNOME/MATE will still try; turn it off at the desktop layer too
└─$ gsettings set org.gnome.desktop.media-handling automount false
└─$ gsettings set org.gnome.desktop.media-handling automount-open false
└─$ gsettings set org.mate.media-handling automount false

# --- make sure nothing on evidence media ever becomes swap ---
└─$ swapon --show                    # should list only your own swap, by UUID
└─# swapoff -a                       # during acquisition/intake, simply have none

# verify /etc/fstab pins your own filesystems by UUID, never by /dev/sdX —
# device names reorder when you attach evidence and you do not want a surprise
└─$ grep -v '^#' /etc/fstab
```

> ▲ **DEVIL — THE SILENT KILLER IS udev, NOT YOU**
>
> The classic contamination event is not a careless `mount`. It is plugging the drive in while a desktop session is running and a udev rule fires before you have typed anything. By the time you notice, the APFS container has been probed, an HFS+ journal may have been replayed, and your acquisition hash no longer matches. Disable automount *before* the drive is ever connected, and verify with `udevadm monitor` that nothing acts on hotplug.
>
> ```
> └─# udevadm monitor --property --subsystem-match=block
> # attach the drive; you want to see detection events and nothing that mounts
> ```

3.3 Base toolchain

Get the standard Debian forensics stack in place. Parrot ships much of this already; the command is idempotent and will simply confirm what is present. The full curated list, including everything that must be built from source, is Appendix A.

```
└─ # apt update && apt install -y \
    sleuthkit autopsy libtsk-dev \
    ewf-tools libewf-dev afflib-tools \
    ddrescue dc3dd dcfldd guymager \
    testdisk foremost scalpel bulk-extractor binwalk magicrescue \
    hashdeep sqlite3 sqlitebrowser python3-pip python3-venv \
    libfuse3-dev fuse3 cmake build-essential git pkg-config \
    libbz2-dev zlib1g-dev libssl-dev libplist-utils \
    exiftool yara radare2 xxd ent
```

● NOTE — WHAT IS NOT IN THE REPOSITORIES

Debian and Parrot package none of the APFS access engines and none of the macOS artifact parsers. `apfs-fuse`, `libfsapfs`, `linux-apfs-rw`, `mac_apt`, the unified log parsers, `chainbreaker` and APOLLO all come from source or from PyPI. That work is §7 and Appendix A.

3.4 Workstation sanity

Disk space	Budget 3× the image size. A working copy, plus carved output, plus a plaso store, plus exported artifacts adds up faster than expected.
RAM	16 GB is a working minimum once plaso, Timesketch or `bulk_extractor` are running. APFS B-tree walks over a large container are memory-hungry.
Time	Set the workstation clock to UTC and know it is correct. Every timestamp you report is relative to it.
Network	Air-gap if the case demands it. Note that `mac_apt`, plaso and pip installs all want the network — do the installs first, then isolate.

```
# put the analysis box in UTC so nothing needs mental arithmetic later
└─ # timedatectl set-timezone UTC
└─ $ timedatectl
               Time zone: UTC (UTC, +0000)
System clock synchronized: yes
```

4 Write-blocking and safe mounting

Three layers, in ascending order of trust: filesystem mount options, block-device read-only flags, and a copy-on-write overlay that makes writes physically impossible to reach the evidence. Use the last one whenever a tool insists on write access.

4.1 Read-only loop devices

The standard way to expose a raw image as a block device. The `-r` is not advisory — the kernel refuses writes to the loop device itself.

```
# attach read-only, scanning for partitions (GPT on a Mac image)
└──╼ # losetup --read-only --partscan --find --show /evidence/mbp-001.dd
/dev/loop0

└──╼ $ lsblk /dev/loop0
NAME      SIZE RO TYPE MOUNTPOINTS
loop0   460.4G  1 loop
├─loop0p1 500M  1 part
└─loop0p2 459.9G 1 part

# RO=1 in that listing is the thing to confirm. Belt and braces:
└──╼ # blockdev --setro /dev/loop0
└──╼ $ blockdev --getro /dev/loop0
1
```

> ● **NOTE — THE APPLE PARTITION LAYOUT**
>
> `loop0p1` is the EFI system partition (FAT32, ~500 MB, holds firmware payloads and boot loaders). `loop0p2` is the **APFS container** — a single GPT partition holding all five or more volumes. Linux will not recognise it without one of the engines in §6; `lsblk` showing an unidentified partition of nearly the whole disk is normal and correct.

4.2 Mount options that do not lie

```
# the full paranoid set, for any filesystem Linux mounts natively (e.g. the EFI partition)
└──╼ # mount -o ro,noexec,nodev,nosuid,noatime,noload /dev/loop0p1 /mnt/efi
```

`ro`	read-only at the VFS layer
`noatime`	do not update access times — meaningless on a `ro` mount but harmless insurance
`noexec,nodev,nosuid`	you are mounting a hostile filesystem; do not let it execute, and do not honour device nodes or setuid bits it contains
`noload`	ext-family only: do not replay the journal. The equivalent trap on HFS+ is journal replay, which genuinely modifies the volume.

4.3 The copy-on-write overlay

Sooner or later a tool demands a writable device — a filesystem checker, a repair utility, a kernel module that mounts read-write only. The answer is never to give it the evidence. Build a device-mapper snapshot: reads come from the evidence, writes land in a throwaway sparse file.

```
# 1. read-only loop over the evidence
└─# EVID=$(losetup --read-only --find --show /evidence/mbp-001.dd)

# 2. a sparse file to absorb every write — costs nothing until used
└─# truncate -s 64G /var/tmp/cow.img
└─# COW=$(losetup --find --show /var/tmp/cow.img)

# 3. stitch them into a snapshot target the same size as the evidence
└─# SZ=$(blockdev --getsz $EVID)
└─# echo "0 $SZ snapshot $EVID $COW P 8" | dmsetup create evid-cow

└─$ lsblk /dev/mapper/evid-cow
NAME      SIZE RO TYPE
evid-cow 460.4G  0 dm      # RO=0 — writable, and the image underneath cannot be touched
```

Point every destructive-looking tool at `/dev/mapper/evid-cow`. When finished, tear it down and the writes evaporate:

```
└─# dmsetup remove evid-cow && losetup -d $COW $EVID && rm -f /var/tmp/cow.img
# then prove the master is untouched
└─$ sha256sum -c /evidence/mbp-001.dd.sha256
/evidence/mbp-001.dd: OK
```

> **▸ TIP — THIS IS ALSO AN EXPERIMENT HARNESS**
>
> The overlay is not only a safety device. It lets you deliberately break things to learn how they behave: run `apfsck` in repair mode, mount read-write with the kernel module, force a journal replay, watch what a snapshot delete does to the free-space bitmap. Tear down, rebuild, try again. Doing this on a copy of the actual evidence — rather than a synthetic test image — is the fastest way to develop reliable intuition about what the filesystem does under stress.

4.4 Verifying you have not written

Re-hashing a 500 GB image after every step is impractical. Two cheaper checks:

```
# the master is immutable and read-only — confirm the flags still hold
└─$ lsattr /evidence/mbp-001.dd; stat -c '%a %U' /evidence/mbp-001.dd
----i---------e------- /evidence/mbp-001.dd
444 root

# mtime should never move after intake
└─$ stat -c 'modified: %y' /evidence/mbp-001.dd
modified: 2026-08-01 14:22:07.000000000 +0000
```

Then re-hash in full at the close of the examination, once, and put both numbers in the report.

5 Case structure, session logging, chain of custody

5.1 A layout that survives contact with a long case

```
~/cases/001/
├── 00-admin/          authorisation, scope, custody forms, the acquisition record
├── 01-image/          hashes, ewfinfo output, partition maps — not the image itself
├── 02-mount/          mountpoints: data/ system/ preboot/ recovery/ snap-*/
├── 03-extract/        artifacts pulled off the image, mirroring their original paths
│   ├── logs/            .tracev3, uuidtext, Install.log, audit
│   ├── db/              knowledgeC, TCC, quarantine, chat.db, History.db
│   ├── plist/           LaunchAgents, LaunchDaemons, BTM, profiles
│   └── user/            per-user home artifacts
├── 04-timeline/       body files, mactime output, plaso storage, super-timeline CSV
├── 05-carve/          photorec/foremost/bulk_extractor output — huge, keep separate
├── 06-analysis/       your own scripts, notes, intermediate findings
└── 07-report/         the deliverable, exhibits, hash manifests
```

▸ TIP — MIRROR THE ORIGINAL PATHS

When you extract an artifact, keep its full original path under `03-extract/`. A file called `TCC.db` in a flat directory is ambiguous — there are at least two, and which one it came from is the entire point. `03-extract/db/Library/Application Support/com.apple.TCC/TCC.db` answers the question by existing.

5.2 Record every command you run

Contemporaneous notes are what make findings defensible. The cheapest possible version is `script`, which records an entire terminal session with timing.

```
# start a recorded session; -T captures timing so it can be replayed
└─▸ $ script -T ~/cases/001/notes/session-$(date -u +%Y%m%dT%H%M%SZ).time \
       -a ~/cases/001/notes/session-$(date -u +%Y%m%dT%H%M%SZ).log

# ... work ...
└─▸ $ exit

# replay it later at original speed, or instantly
└─▸ $ scriptreplay -T session.time session.log

# nicer alternative if installed — produces a shareable asciicast
└─▸ $ asciinema rec ~/cases/001/notes/mount-attempt.cast
```

▸ TIP — TIMESTAMP YOUR SHELL HISTORY

Add this to `~/.bashrc` on the analysis box. It costs nothing and turns your history file into a rough audit log.

```
export HISTTIMEFORMAT='%F %T UTC  '
export HISTSIZE=100000 HISTFILESIZE=100000
export PROMPT_COMMAND='history -a'   # flush after every command, not at exit
```

5.3 Hash manifests for everything you produce

Every artifact you extract and every file you deliver gets hashed, so that a year from now you can prove the file in the report is the file from the image.

```
# recursive manifest of extracted artifacts
└─╼ $ hashdeep -c sha256 -r ~/cases/001/03-extract > ~/cases/001/01-image/extract.manifest

# later — audit against the manifest; reports moved, changed and new files
└─╼ $ hashdeep -c sha256 -r -a -k ~/cases/001/01-image/extract.manifest ~/cases/001/03-extract
hashdeep: Audit passed
```

5.4 The custody record

Minimum viable chain of custody for a single-examiner case. Keep it in `00-admin/` as plain text so it is diffable and never becomes unreadable.

Field	Content
Case reference	Internal identifier, and the authority under which you are working
Exhibit	Device make, model, serial, storage size; the exhibit number it carries
Acquired	UTC timestamp, method (recoveryOS + `dd`), examiner, tool versions
Source hash	SHA-256 of `/dev/rdisk0` as read on the subject machine
Image hash	SHA-256 of the resulting file, and every subsequent verification with its date
Alterations	**SIP disabled at <time> by examiner** — and anything else you knowingly changed
Transfers	Who held the media, when, and in what condition
Analysis env	Parrot version, kernel, and the version of every parser used for a reported finding

■ CAUTION — RECORD TOOL VERSIONS AS YOU GO

The APFS parsers in this manual are young, move quickly, and disagree with each other. A finding produced by `apfs-fuse` at one commit may not reproduce at another. Capture versions at the time of use — `git rev-parse --short HEAD` in each source tree, `pip freeze` for the Python tools — and store that alongside the finding. "It worked when I ran it" is not a defence you want to have to make.

III

OPENING APFS ON LINUX

Linux has no native APFS support. What it has instead is four independent reimplementations, written by different people, with different blind spots, none of which is complete. Knowing which one to reach for — and where each one quietly lies to you — is the single most useful thing in this manual.

06 The four access engines
07 Building the toolchain on Parrot
08 Container discovery and layout
09 FileVault2 — keybags, wrapped keys, unlocking
10 Engine capability matrix and known gotchas

6 The four access engines

Every APFS technique in this manual runs on top of one of four independent implementations. They are not interchangeable. Learn the shape of each.

6.1 libyal `libfsapfs` — the forensic reference

Joachim Metz's library, part of the libyal family that also gives you `libewf`. It is read-only by design, exposes container and volume metadata in exhaustive detail, handles FileVault with a password or recovery key, and — crucially — is the APFS backend that **plaso** uses through dfVFS. If your timeline tooling can see a file, it is usually seeing it through this library.

Strengths	Metadata depth, encryption support, snapshot enumeration, stable behaviour, integrates with plaso
Weaknesses	Slower on large trees; FUSE mount is single-threaded; less forgiving of damaged containers
Binaries	`fsapfsinfo` (inspect), `fsapfsmount` (FUSE mount)
Use it for	Ground truth about the container, encrypted volumes, and anything feeding plaso

6.2 `apfs-fuse` — the practical workhorse

Simon Gander's implementation. Read-only FUSE, fast, tolerant, and the best of the four at the things you actually spend your day doing: transparent `decmpfs`/LZFSE decompression, extended attributes, and mounting individual snapshots by name.

Strengths	Speed, compression handling, xattrs, snapshot mounting, FileVault support, copes with odd containers
Weaknesses	Less metadata introspection than libfsapfs; occasional divergence on exotic records
Binaries	`apfs-fuse`, `apfsutil`
Use it for	Day-to-day file access, snapshot work, anything involving compressed files

6.3 `linux-apfs-oot` — the kernel module

Ernesto A. Fernández's out-of-tree kernel driver, with its companion `apfsprogs` (`apfsck`, `apfs-snap`, `apfs-label`). Being a real kernel filesystem it is dramatically faster than FUSE and works with tools that dislike FUSE mounts. It is also the only one that can write.

Strengths	Kernel speed, real `mount` semantics, `apfsck` structural verification, snapshot mounting
Weaknesses	No FileVault support at all. Write support is experimental. Needs DKMS against your running kernel.
Use it for	Fast bulk access to unencrypted volumes, and structural integrity checking

> **■ CAUTION — THE MODULE CAN WRITE**
>
> This is the one engine capable of modifying evidence. Mount it `ro` explicitly, always, and if you have any reason to exercise its write path, do it against the copy-on-write overlay from §4.3 and never against the image.

6.4 The Sleuth Kit — pool-aware, classically shaped

TSK gained APFS support through its pool abstraction: an APFS container is a pool, and the volumes inside it are addressed by a pool volume block. This buys you the entire classical toolchain — `fls`, `istat`, `icat`, `tsk_recover`, body files for `mactime` — against APFS.

Strengths	The whole traditional workflow; inode-level addressing; body-file timelines; Autopsy integration
Weaknesses	Command-line encrypted-volume handling is inconsistent between builds; no snapshot support; unallocated-space semantics do not map cleanly onto copy-on-write
Use it for	Timelines, inode-level work, deleted-entry recovery on unencrypted volumes

▲ DEVIL — THE ENCRYPTION / BLOCK-LEVEL DEADLOCK

This is the defining constraint of your case and it is worth internalising now. **There is no reliable tool on Linux that decrypts a FileVault APFS volume back out to a flat raw image.** `apfs-fuse` and `fsapfsmount` decrypt on the fly, at the file level, and present you a filesystem. They do not hand you a decrypted block device.

The consequence: for your encrypted Data volume you get excellent **file-level** access, but block-level techniques — TSK inode walks, `blkls` unallocated extraction, raw carving of free space — either do not work or must be run through the FUSE mount, where they see only allocated files. Plan the examination around this. The unencrypted volumes (Preboot, Recovery, and on many systems the sealed System volume) remain fully available to block-level tooling, which is one reason §49 matters more than its size suggests.

7 Building the toolchain on Parrot

None of the four are packaged. All four build cleanly on current Debian/Parrot. Do these once, record the commit hashes, and move on.

7.1 Prerequisites

```
└─# apt install -y build-essential cmake git pkg-config autoconf automake libtool \
    libfuse3-dev fuse3 libbz2-dev zlib1g-dev libssl-dev libattr1-dev \
    python3-dev python3-pip flex bison dkms linux-headers-$(uname -r)
```

7.2 `apfs-fuse`

```
└─$ git clone https://github.com/sgan81/apfs-fuse.git ~/tools/apfs-fuse
└─$ cd ~/tools/apfs-fuse && git submodule update --init   # pulls the bundled lzfse
└─$ mkdir build && cd build
└─$ cmake .. -DUSE_FUSE3=ON && make -j$(nproc)
└─# make install && ldconfig

# record what you built — this goes in the case notes
└─$ git -C ~/tools/apfs-fuse rev-parse --short HEAD
└─$ apfs-fuse --help | head -20
```

7.3 libyal `libfsapfs`

```
└─$ git clone https://github.com/libyal/libfsapfs.git ~/tools/libfsapfs
└─$ cd ~/tools/libfsapfs
└─$ ./synclibs.sh          # fetches the sibling libyal dependencies
└─$ ./autogen.sh
└─$ ./configure --enable-python --with-libfuse
└─$ make -j$(nproc)
└─# make install && ldconfig

└─$ fsapfsinfo -V
fsapfsinfo 20240508
```

▸ TIP — THE PYTHON BINDINGS ARE THE REAL PRIZE

`--enable-python` builds `pyfsapfs`, which lets you walk the container programmatically instead of scraping CLI output. Nearly every custom script in §62 and the exotic techniques in Part IX are far easier through it.

```
└─$ python3 -c "import pyfsapfs; print(pyfsapfs.get_version())"
20240508
```

7.4 The kernel module and `apfsprogs`

```
# --- out-of-tree read/write driver ---
└─╼ $ git clone https://github.com/linux-apfs/linux-apfs-oot.git ~/tools/linux-apfs-oot
└─╼ $ cd ~/tools/linux-apfs-oot && make -j$(nproc)
└─╼ # make install && depmod -a && modprobe apfs
└─╼ $ lsmod | grep apfs
apfs                  282624  0

# --- apfsck / apfs-snap / apfs-label ---
└─╼ $ git clone https://github.com/linux-apfs/apfsprogs.git ~/tools/apfsprogs
└─╼ $ cd ~/tools/apfsprogs/apfsck && make -j$(nproc) && sudo make install
└─╼ $ cd ../apfs-snap && make -j$(nproc) && sudo make install
└─╼ $ cd ../apfs-label && make -j$(nproc) && sudo make install
```

● NOTE — DKMS SO IT SURVIVES KERNEL UPDATES

A rolling distribution will update your kernel and silently break the module. Register it with DKMS so it rebuilds automatically, or you will one day find your mount command failing mid-case for no apparent reason.

7.5 The Sleuth Kit with pool support

Parrot's packaged `sleuthkit` is generally recent enough. Confirm APFS is compiled in before trusting it — a build without pool support fails in confusing ways rather than saying so.

```
└─╼ $ fls -V; pstat -V 2>/dev/null || echo "no pool support — build from source"
The Sleuth Kit ver 4.12.1

# if pstat is missing, build from source with the pool framework
└─╼ $ git clone https://github.com/sleuthkit/sleuthkit.git ~/tools/sleuthkit
└─╼ $ cd ~/tools/sleuthkit && ./bootstrap && ./configure && make -j$(nproc)
└─╼ # make install && ldconfig
```

8 Container discovery and layout

Before unlocking anything, map what is there. Three tools give three different views and you want all three.

8.1 The partition table

```
└─╼ $ mmls /evidence/mbp-001.dd
GUID Partition Table (EFI)
Offset Sector: 0
Units are in 512-byte sectors

      Slot      Start        End          Length       Description
000:  Meta      0000000000   0000000000   0000000001   Safety Table
001:  -------   0000000000   0000000039   0000000040   Unallocated
002:  Meta      0000000001   0000000001   0000000001   GPT Header
003:  Meta      0000000002   0000000033   0000000032   Partition Table
004:  000       0000000040   0001024039   0001024000   EFI System Partition
005:  001       0001024040   0965578399   0964554360   Apple APFS Container
```

Note the container start sector — **1024040** here. Multiply by 512 for the byte offset that libyal tools want: `524308480`. TSK wants the sector number via `-o 1024040`.

8.2 The container and its volumes

```
└─╼ $ fsapfsinfo -o 524308480 /evidence/mbp-001.dd
fsapfsinfo 20240508

Apple File System (APFS) information:

Container information:
        Identifier                : 8f2a1c40-3b7e-4d19-9c22-5a6b0e1d77f3
        Number of blocks          : 120569295
        Block size                : 4096 bytes
        Number of volumes         : 5

Volume: 1 information:
        Identifier                : 1a3f...  Name: Macintosh HD
        Compatible features       : sealed volume
        Is encrypted              : no
Volume: 2 information:
        Name: Preboot           Is encrypted: no
Volume: 3 information:
        Name: Recovery          Is encrypted: no
Volume: 4 information:
        Name: VM                Is encrypted: no
Volume: 5 information:
        Name: Macintosh HD - Data   Is encrypted: yes
```

● NOTE — THE STANDARD FIVE-VOLUME LAYOUT 13+

Every modern Mac looks like this. Learn it once:

Volume	Role	What it holds and why you care
Macintosh HD	System	Sealed, read-only, Apple-signed. The OS itself. Its value is integrity — a broken seal means tampering (§46).
Macintosh HD - Data	Data	Everything that matters. All user data, all logs, all persistence. FileVault-encrypted. This is your target.
Preboot	Preboot	Per-volume boot support, LocalPolicy, ownership records. Unencrypted and routinely ignored — see §49.
Recovery	Recovery	recoveryOS image. Unencrypted. Rarely modified, which makes modification interesting.
VM	VM	Swap files and the hibernation image. Unencrypted container-side but its contents may be encrypted (§50).

8.3 The engines' own views

```
# apfs-fuse's container lister — fast, includes snapshot counts
└──╼ $ apfsutil /evidence/mbp-001.dd

# TSK's pool view — gives you the pool volume block you need for fls/istat
└──╼ $ pstat -o 1024040 /evidence/mbp-001.dd
POOL CONTAINER INFORMATION
----------------------------------------
Type: APFS
Block Size: 4096
Number of Volumes: 5

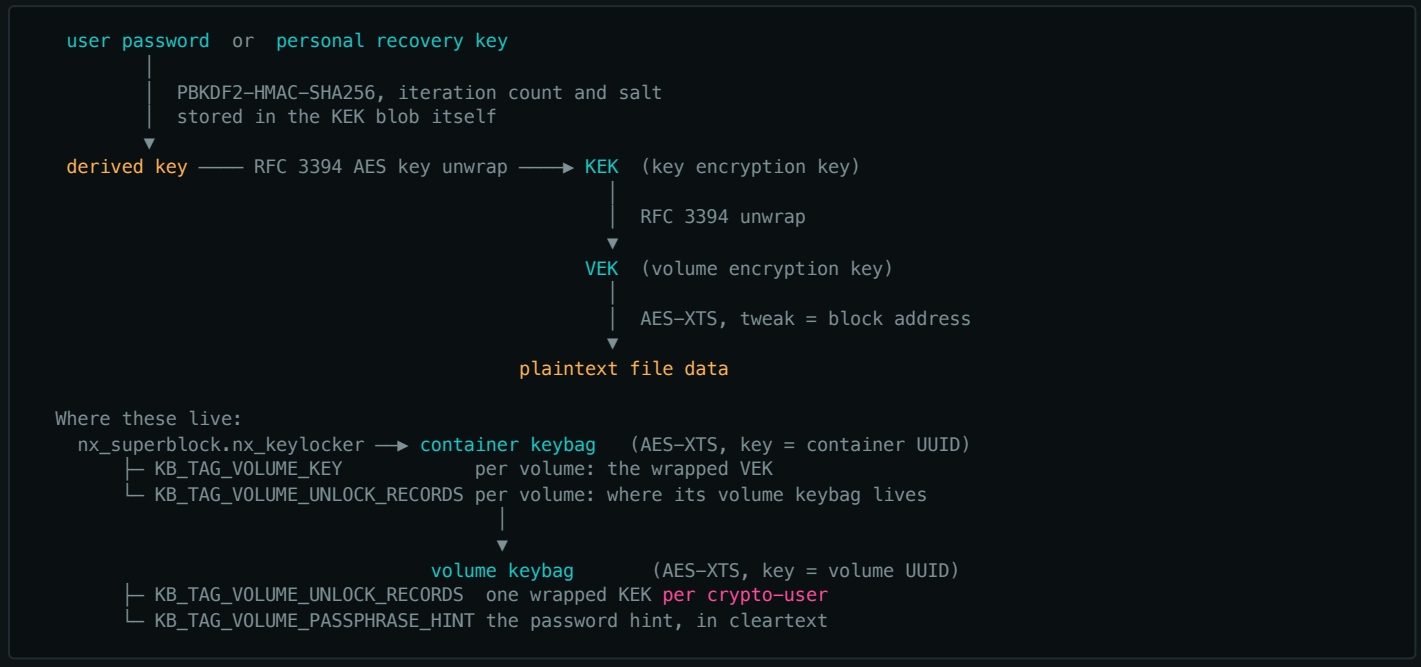
VOLUME INFORMATION
----------------------------------------
Volume APSB Block: 1102
  Name: Macintosh HD
  Role: System
Volume APSB Block: 1276
  Name: Macintosh HD - Data
  Role: Data
  Encrypted: Yes
```

The **APSB block** is the address you pass to TSK as `-B`. Write these down; every TSK command in Part V needs one.

9 FileVault2 — keybags, wrapped keys, unlocking

You have the password. This section explains what that password actually unlocks, because the chain has four links and knowing them tells you what can go wrong and what else is recoverable.

9.1 The key hierarchy



▲ DEVIL — THE KEYBAGS ARE "ENCRYPTED" WITH PUBLIC INFORMATION

Both keybags are encrypted with AES-XTS using the container UUID and the volume UUID respectively — values that sit in cleartext in the superblocks a few hundred bytes away. This is obfuscation, not security, and it is deliberate: it lets the boot process locate unlock records before any secret is known. For you it means **the keybag structure is fully readable without the password**. You can enumerate how many crypto-users exist, read the password hint, and see whether an institutional or personal recovery key was enrolled — all before authenticating. That is real intelligence about the account even in the case where you never get the password.

9.2 Unlocking with libfsapfs

```
# inspect first: which volume is encrypted, and does it take our password
└─╼ $ fsapfsinfo -o 524308480 -p 'TheUserPassword' -f 5 /evidence/mbp-001.dd

# mount every volume the password opens, read-only, under /mnt/apfs
└─╼ # mkdir -p /mnt/apfs
└─╼ $ fsapfsmount -o 524308480 -p 'TheUserPassword' /evidence/mbp-001.dd /mnt/apfs

└─╼ $ ls /mnt/apfs/
apfs1  apfs2  apfs3  apfs4  apfs5      # one directory per volume, in container order
└─╼ $ ls /mnt/apfs/apfs5/
Applications  Library  System  Users  private  usr  ...
```

`-o <bytes>`	byte offset of the container (sector × 512)
`-p <pw>`	user password
`-r <key>`	personal recovery key, the 24-character dashed string, if you have that instead
`-f <n>`	restrict to one volume by index
`-X allow_other`	let other users (and root-run tools) see the mount

9.3 Unlocking with apfs-fuse

```
# attach the image as a read-only loop device first
└─# LOOP=$(losetup --read-only --partscan --find --show /evidence/mbp-001.dd)

# list volumes and their indices
└─$ apfs-fuse -l ${LOOP}p2

# mount volume 4 (0-indexed — the Data volume) with the password
└─# apfs-fuse -v 4 -p 'TheUserPassword' -o ro,allow_other ${LOOP}p2 /mnt/data

└─$ ls /mnt/data/Users/
Shared  jdoe

# unmount cleanly when done
└─# fusermount3 -u /mnt/data
```

■ CAUTION — PASSWORDS ON THE COMMAND LINE

`-p 'password'` puts the subject's credential into your shell history, into your `script` session log, and into `/proc` where any local process can read it while the command runs. On a shared or monitored analysis box this is a genuine disclosure. Prefer an interactive prompt where the tool offers one, and if you must pass it inline, prefix the command with a space so `HISTCONTROL=ignorespace` keeps it out of history — and remember to scrub the session transcript before it becomes an exhibit.

9.4 If you did not have the password

Out of scope for your case, but worth a paragraph because it explains the shape of the problem. The wrapped KEK blob — salt, iteration count, and the wrapped key — is extractable from the volume keybag without authenticating, and is a legitimate offline attack target. `hashcat` mode **18300** ("Apple File System / APFS") consumes exactly that structure. The extraction is done by `apfs2hashcat.py`, distributed with hashcat's tools.

The practical reality is that PBKDF2 iteration counts on modern macOS make this viable only against weak or known-pattern passwords. Where it does pay off is with the **personal recovery key**, whose format is fixed and whose keyspace, while large, is structured.

9.5 Confirming you are actually decrypted

A silently-failed unlock can present as an empty or nonsensical filesystem rather than an error. Check positively:

```
# a known-plaintext file that exists on every macOS install
└─$ file /mnt/apfs/apfs5/private/var/log/install.log
/mnt/apfs/apfs5/private/var/log/install.log: ASCII text

# and the system version, which also tells you exactly which OS you are dealing with
└─$ plistutil -i /mnt/apfs/apfs1/System/Library/CoreServices/SystemVersion.plist -o - | \
    grep -A1 'ProductVersion\|ProductBuildVersion'
<key>ProductBuildVersion</key>   <string>23H124</string>
<key>ProductVersion</key>        <string>14.7.1</string>
```

▸ TIP — PIN THE OS VERSION BEFORE ANYTHING ELSE

Read `SystemVersion.plist` as the first thing you do after mounting, and write the build number at the top of your notes. Half the artifact paths in Part VII moved between releases. Every version tag in this manual is meaningless until you know which number you are holding.

10 Engine capability matrix and known gotchas

Capability	libfsapfs	apfs-fuse	linux-apfs-oot	Sleuth Kit
Read unencrypted volume	yes	yes	yes	yes
FileVault2 (password)	**yes**	**yes**	~~no~~	build-dependent
FileVault2 (recovery key)	yes `-r`	partial	~~no~~	~~no~~
Mount a named snapshot	yes	yes `-s`	yes `-o snap=`	~~no~~
Enumerate snapshots	yes	yes	yes	~~no~~
`decmpfs` / LZFSE transparent	partial	**yes**	yes	partial
Extended attributes	yes	**yes**	yes	yes
Sparse files / clones	yes	yes	yes	partial
Sealed volume awareness	reports	reports	reports	~~no~~
Fusion (tier2) containers	yes	yes	partial	partial
Inode-level addressing	via API	~~no~~	~~no~~	**yes**
Body file / `mactime`	via plaso	~~no~~	~~no~~	**yes**
Structural integrity check	partial	~~no~~	`apfsck`	~~no~~
Write capability	never	never	~~experimental~~	never

10.1 Gotchas worth knowing before they cost you a day

▲ DEVIL — FUSE MOUNTS HIDE THE FILESYSTEM FROM YOUR TOOLS

A FUSE mount presents files. It does not present a block device, inode numbers you can trust across tools, or unallocated space. Anything in Part V that addresses the filesystem structurally — `fls`, `istat`, `blkls`, `tsk_recover` — must run against the `image`, not against `/mnt/apfs`. Running `fls` on a FUSE mount-point will either error or silently describe your own Linux filesystem instead. This mistake is common and produces findings that are quietly, completely wrong.

▲ DEVIL — THE ENGINES DISAGREE, AND THAT IS DATA

When `apfs-fuse` and `fsapfsinfo` report different things about the same object — a different size, a file one sees and the other does not, a differing timestamp — do not simply pick the one you like. Divergence usually means one of three things: a genuine parser bug, a structurally unusual record (an oversized xattr, a clone, a sparse extent), or **deliberate manipulation designed to confuse parsers**. Record the disagreement and investigate it. In several published cases the discrepancy `was` the finding.

● NOTE — CASE SENSITIVITY

The Data volume on macOS is case-`insensitive` but case-`preserving`. Your Linux filesystem is case-sensitive. Two files that cannot coexist on macOS can coexist in your extraction directory, and a script that assumes `Library` and `library` are distinct will behave differently on the mount than the Mac did. When matching paths, compare case-insensitively.

● NOTE — NORMALISATION

APFS stores filenames as UTF-8 and, unlike HFS+, does `not` normalise them — but it does hash them case-insensitively for lookup on case-insensitive volumes. The upshot: two filenames that look identical on screen can differ in byte representation (precomposed vs decomposed accents). When grepping for a filename containing non-ASCII characters, search for both normalisation forms or you will miss it.

```
└──╼ $ python3 -c "import unicodedata as u; n='café.txt'; \
print(repr(u.normalize('NFC',n)), repr(u.normalize('NFD',n)))"
'café.txt' 'café.txt'
```

IV

APFS INTERNALS THAT MATTER

Not a specification walkthrough. This is the subset of APFS structure that changes what you conclude: where old states of the filesystem survive, why a deleted file may still be fully intact, which timestamp moved and why, and why the path you think you are looking at is not the path macOS was using.

11 Container anatomy — NXSB, checkpoints, transaction IDs
12 Volumes, roles, firmlinks and the System/Data split
13 The catalog B-tree and its record types
14 Time in APFS — four timestamps and what moves them
15 Clones, snapshots, tombstones and copy-on-write
16 Sealed System Volume and cryptexes

11 Container anatomy

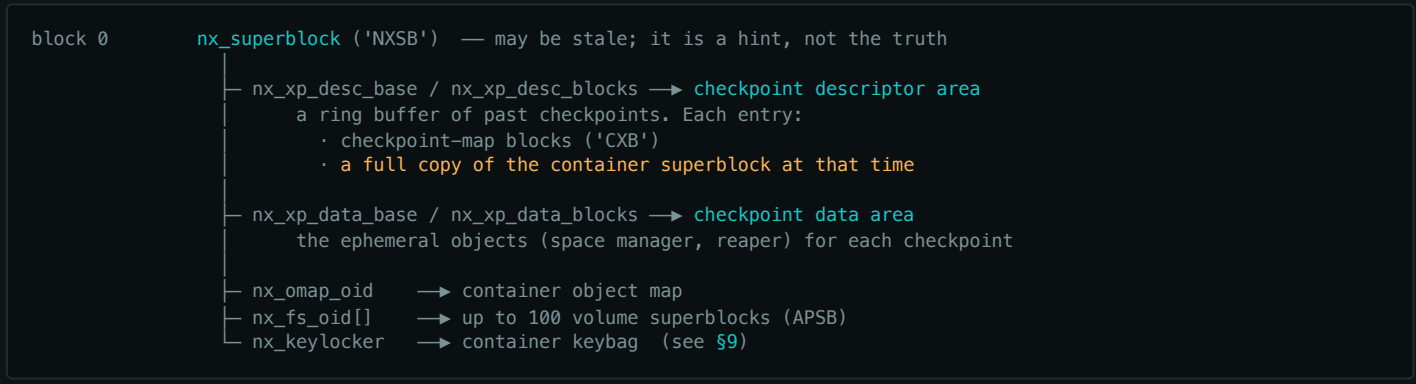
11.1 Every object carries its own provenance

APFS objects begin with a 32-byte `obj_phys` header. Four of its fields are directly useful to you:

o_cksum	Fletcher-64 over the rest of the block. A block whose checksum fails is either corrupt or forged — and you can test this yourself, which matters in §54.
o_oid	Object identifier. For virtual objects this is looked up through the object map.
o_xid	Transaction identifier. The single most under-used forensic field in APFS. Monotonically increasing, per container, for the life of the filesystem.
o_type / o_subtype	What the object is, and what a B-tree contains.

> **▲ DEVIL — THE XID IS A CLOCK NOBODY CAN BACKDATE**
>
> Timestamps are file metadata and are trivially rewritten. The XID is not: it is assigned by the filesystem when a transaction commits, always increases, and is not exposed through any API an attacker can reach. This gives you an **independent, monotonic ordering of every write the filesystem ever made** — one that cannot be forged without rewriting the container wholesale.
>
> A file claiming a creation date of 2019 sitting in a structure with an XID adjacent to files written last Tuesday is a backdated file. This is the foundation of §43 and it is the most reliable anti-timestomping technique available on this filesystem.

11.2 The container superblock and the checkpoint areas



Mounting picks the superblock in the descriptor area with the **highest XID and a valid checksum**. Block 0 is only a starting point.

> **▲ DEVIL — THE DESCRIPTOR AREA IS A TIME MACHINE**
>
> The descriptor ring holds superseded container superblocks — complete, self-consistent snapshots of the filesystem's root structure as it existed at earlier transactions. Because APFS is copy-on-write, the object trees those old superblocks point at are frequently **still on disk and still intact**, simply no longer referenced by the current state.
>
> This means you can often mount, or at least walk, the filesystem as it was several checkpoints ago — recovering files deleted after the last snapshot but before the present. No snapshot required, no Time Machine required. Almost no tooling does this automatically. §41 is entirely about doing it by hand.

```
# find every container superblock in the image by magic, not just the one at block 0
└─╼ $ xxd -c 32 /evidence/mbp-001.dd | grep -i 'NXSB' | head -40

# cleaner: scan block-aligned offsets for the NXSB magic at offset 0x20 of the block
└─╼ $ python3 - <<'PY'
import sys
BS=4096; MAGIC=b'NXSB'
with open('/evidence/mbp-001.dd','rb') as f:
    f.seek(524308480)                       # container start
    for blk in range(0, 4096):              # descriptor area is early in the container
        d = f.read(BS)
        if not d: break
        if d[32:36] == MAGIC:
            xid = int.from_bytes(d[16:24],'little')
            print(f"block {blk:6d}  xid {xid}")
PY
block      0  xid 214417
block    193  xid 214414
block    194  xid 214415
block    195  xid 214416
block    196  xid 214417      <-- newest valid checkpoint
```

11.3 The object map

APFS separates virtual object IDs from physical block addresses. The object map (omap) is a B-tree keyed on `(oid, xid)` that resolves an object ID to the block holding its content **as of a given transaction**.

That `xid` in the key is the mechanism behind snapshots: the same object ID resolves to different physical blocks depending on which transaction you ask about. Ask for an old XID and you get the old data — if it has not been reclaimed.

11.4 The space manager

Tracks free and allocated blocks via allocation bitmaps. Forensically it answers one question that matters enormously: **which blocks are currently unreferenced?** Unreferenced blocks on a copy-on-write filesystem are where superseded versions of files live. §45 covers extracting them.

12 Volumes, roles, firmlinks and the System/Data split

12.1 The volume superblock records who built it

Each volume's `apfs_superblock` ('APSB') carries two fields that are pure forensic value and are almost never mentioned:

`apfs_formatted_by`	The macOS build string that created this volume, plus a timestamp and the XID at the time. Tells you when the machine was set up or last erased.
`apfs_modified_by[8]`	A rolling list of the last eight OS builds that mounted this volume writable, each with a timestamp and XID.

▲ DEVIL — apfs_modified_by IS AN OS-LEVEL ACCESS LOG

Eight entries, each naming a specific macOS build and when it touched the volume. Read it and you learn the machine's update history — and, far more interestingly, **whether the volume was ever mounted by a different system than the one installed on it.** A recoveryOS build, an installer build, or an OS version that was never installed on this Mac appearing in that list means the disk was mounted somewhere else. That is evidence of exactly the kind of offline tampering the rest of your examination struggles to see.

Your own acquisition may appear here too — recoveryOS mounting the volume. Check the timestamps against your acquisition record before drawing conclusions.

```
└──╼ $ fsapfsinfo -o 524308480 -p 'PW' -f 5 /evidence/mbp-001.dd | \
    grep -A3 -i 'formatted by\|modified by'
```

12.2 Volume roles

Role	Value	Notes
NONE	0x0000	Plain data volume; also what a manually created volume gets
SYSTEM	0x0001	Sealed OS volume `13+`
USER	0x0002	Legacy home-directory role
RECOVERY	0x0004	recoveryOS
VM	0x0008	Swap and hibernation — see §50
PREBOOT	0x0010	Boot support, LocalPolicy — see §49
INSTALLER	0x0020	Present during OS installation; lingering ones are interesting
DATA	0x0040	The Data volume — everything you care about
BACKUP	0x0100	Time Machine local destination
UPDATE / XART / HARDWARE	varies	Firmware and secure-storage roles; rarely relevant, always worth noting if unexpected

● NOTE — AN EXTRA VOLUME IS ALWAYS WORTH A LOOK

The standard layout is five volumes. A sixth with role NONE is not inherently suspicious — people create volumes for legitimate reasons — but it is unusual enough that it should never go unexamined. An attacker who wants storage that survives a user wiping their home directory, and that most triage tools will not enumerate, can simply add a volume to the container.

12.3 Firmlinks — why your paths are wrong

Since Catalina the OS is split across two volumes that macOS presents as one tree. The System volume is read-only and sealed; everything writable lives on the Data volume, mounted at `/System/Volumes/Data`. **Firmlinks** are the bidirectional bindings that make the split invisible: they are neither symlinks nor hard links, and they exist only in APFS.

The mapping lives on the System volume in `/usr/share/firmlinks`:

```
└─╼ $ cat /mnt/apfs/apfs1/usr/share/firmlinks
/AppleInternal          AppleInternal
/Applications           Applications
/Library                Library
/System/Library/Caches  System/Library/Caches
/System/Library/Assets  System/Library/Assets
/System/Library/Speech  System/Library/Speech
/Users                  Users
/Volumes                Volumes
/cores                  cores
/opt                    opt
/private/tmp            private/tmp
/private/var            private/var
/usr/local              usr/local
/usr/libexec/cups       usr/libexec/cups
/usr/share/snmp         usr/share/snmp
```

▲ DEVIL — TRANSLATE EVERY PATH YOU READ IN A GUIDE

Every macOS forensics reference — including the artifact tables in Part VII — writes paths the way the running system sees them: `/var/log/install.log`, `/Library/LaunchDaemons/`. **None of those paths exist on the volume you mounted.** They are firmlinked. On your Linux mount they resolve as:

Path as macOS shows it	Where it actually is
/var/log/install.log	<DATA>/private/var/log/install.log
/Library/LaunchDaemons/	<DATA>/Library/LaunchDaemons/
/Users/jdoe/	<DATA>/Users/jdoe/
/Applications/	<DATA>/Applications/
/System/Library/LaunchDaemons/	<SYS>/System/Library/LaunchDaemons/
/usr/bin/, /bin/, /sbin/	<SYS>/usr/bin/ etc — sealed, read-only
/usr/local/	<DATA>/usr/local/ — **writable, and therefore interesting**
/private/tmp/	<DATA>/private/tmp/

The rule of thumb: **if it is writable, it is on Data.** `/usr/local` is the one that catches people out — it looks like a system path, it sits inside a sealed volume's namespace, and it is fully writable because it is firmlinked to Data. It is a favourite persistence location for exactly that reason.

12.4 Mounting both halves at once

```
# mount System and Data separately, then reason about them together
└─╼ # mkdir -p /mnt/sys /mnt/data
└─╼ $ fsapfsmount -o 524308480 -f 1 /evidence/mbp-001.dd /mnt/sys
└─╼ $ fsapfsmount -o 524308480 -f 5 -p 'PW' /evidence/mbp-001.dd /mnt/data

# a crude but effective reconstruction of the unified view, read-only
└─╼ # mkdir -p /mnt/macos && mount --bind /mnt/sys/apfs1 /mnt/macos
└─╼ # mount --bind /mnt/data/apfs5/Users   /mnt/macos/Users
└─╼ # mount --bind /mnt/data/apfs5/Library /mnt/macos/Library
└─╼ # mount --bind /mnt/data/apfs5/private/var /mnt/macos/private/var
# now /mnt/macos approximates what the Mac itself saw — useful when running
# parsers that expect real macOS paths, e.g. mac_apt in some modes
```

13 The catalog B-tree and its record types

One B-tree per volume ("fsroot") holds all filesystem metadata. Keys are `(object_id, record_type)`, so every record about one file sorts together.

Type	Name	What it gives you
1	SNAP_METADATA	A snapshot's name, XID and tree pointers — §42
2	EXTENT	Physical extent reference counts — how clone sharing is tracked
3	**INODE**	The file record: parent, timestamps, mode, owner, flags, extended fields
4	**XATTR**	Extended attributes — quarantine, provenance, Finder info. §47
5	SIBLING_LINK	Hard link siblings — one inode, several names
6	DSTREAM_ID	Data stream reference counting
7	CRYPTO_STATE	Per-file encryption state and protection class
8	**FILE_EXTENT**	Where the file's bytes physically are — the map for carving
9	**DIR_REC**	A directory entry: name → child object id
10	DIR_STATS	Directory size accounting
11	SNAP_NAME	Snapshot name → XID lookup
12	SIBLING_MAP	Reverse hard-link mapping
13	FILE_INFO	Per-file data hashes on sealed volumes — §46

13.1 The inode record's useful fields

parent_id	Object id of the containing directory — how you rebuild a path from an orphaned inode
private_id	The inode's own id; for files, also the data stream id
create/mod/change/access_time	Four uint64 nanosecond Unix timestamps — §14
internal_flags	Includes `INODE_WAS_EVER_CLONED`, `INODE_IS_SPARSE`, `INODE_HAS_RSRC_FORK`, `INODE_NO_RSRC_FORK`
bsd_flags	BSD file flags — `UF_HIDDEN`, `UF_IMMUTABLE`, `SF_RESTRICTED`. Hidden and immutable files matter.
owner / group / mode	UID, GID, permissions. Cross-reference UIDs against §55 when accounts have been deleted.
write_generation_counter	Increments on every write to the file. A file with an old mtime and a high write counter has been modified more than its timestamps admit.
xfields	Extended fields: the real filename, dstream info, document id, Finder info, sparse byte count, purgeable flags

▲ DEVIL — THE WRITE GENERATION COUNTER

Almost nothing surfaces this field, and no timestamp-manipulation tool resets it, because it is not reachable through any documented API. A file whose `mod_time` claims one write but whose `write_generation_counter` reads 400 has been written 400 times. Pair it with the XID (§43) and you have two independent witnesses against a forged timestamp.

14 Time in APFS

Four timestamps per inode, all uint64 nanoseconds since the Unix epoch — not the Mac absolute time (2001 epoch) you will meet inside Apple's own databases in Part VII. Do not mix them up.

Field	Shorthand	Moves when	Forger's difficulty
create_time	B (birth)	The inode is created. Never updated afterwards by normal operation.	Settable via API
mod_time	M	File contents change	Trivially settable (touch)
change_time	C	File metadata changes — permissions, ownership, xattrs, **and any change to the other timestamps**	**Hard** — see below
access_time	A	File is read. Often unreliable: macOS mounts do update it, but caching and noatime-like behaviour make it lossy.	Settable

▲ DEVIL — CHANGE TIME IS THE ONE THEY FORGET

`change_time` is updated by the kernel whenever inode metadata is touched — including when someone rewrites `mod_time`. There is no supported way to set it directly. So the classic timestomp signature is:

```
create_time  2019-03-14 09:12:00   ← claimed
mod_time     2019-03-14 09:12:00   ← claimed
change_time  2026-07-31 02:44:17   ← when the lie was told
access_time  2019-03-14 09:12:00
```

A `change_time` substantially later than all three others, with no plausible metadata event to explain it, is a timestomp until proven otherwise. Tools that stomp all four exist — but setting `change_time` requires either changing the system clock (which leaves its own tracks in the unified log) or writing to the raw device. Both are noisy. See §53 for how to corroborate.

```
# read all four with nanosecond precision from a mounted volume
└─╼ $ stat --printf='B %w\nM %y\nC %z\nA %x\n' /mnt/data/apfs5/Users/jdoe/Documents/notes.rtf

# fsapfsinfo gives them straight from the inode record, bypassing FUSE translation
└─╼ $ fsapfsinfo -o 524308480 -p 'PW' -f 5 -E 'Users/jdoe/Documents/notes.rtf' \
    /evidence/mbp-001.dd
```

● NOTE — FUSE MAY NOT GIVE YOU BIRTH TIME

`stat`'s `%w` (birth) often reports `-` through a FUSE mount because the FUSE protocol has no universal birth-time field. When the creation time matters — and in timestamp analysis it always matters — read it from `fsapfsinfo` or through `pyfsapfs`, not from `stat`.

15 Clones, snapshots, tombstones and copy-on-write

15.1 Copy-on-write changes what "deleted" means

APFS never overwrites a live block. Modifying a file allocates new blocks and repoints the tree; the old blocks become unreferenced but remain on disk until reused. Two consequences that invert habits formed on NTFS or ext4:

- **Old versions of files persist far longer than expected.** Every save of a document may have left its predecessor intact somewhere in free space.
- **Classical carving underperforms.** Files are fragmented across non-contiguous extents by design, so header/footer carvers reassemble them badly. Extent-aware recovery beats carving on APFS — see §19 and §45.

15.2 Clones

Copying a file in Finder creates a clone: a second inode pointing at the same physical extents, with a reference count. No data is duplicated until one side is written.

Forensically: two files with identical content may share every block, so finding "a copy" of a file tells you nothing about when it was copied unless you look at the inode. The `INODE_WAS_EVER_CLONED` flag records that a clone relationship once existed **even after the other side is deleted** — evidence that a file was duplicated, surviving the deletion of the duplicate.

15.3 Snapshots

A snapshot pins the object map at a given XID, freezing an entire consistent filesystem state. macOS creates them constantly and mostly silently:

Time Machine local	`com.apple.TimeMachine.YYYY-MM-DD-HHMMSS.local` — created hourly when a backup destination is configured, retained ~24h, and created before every OS update
Installer	Taken automatically before a macOS upgrade
Manual / third-party	Backup software, `tmutil localsnapshot`

```
# enumerate every snapshot on the Data volume
└─╼ $ fsapfsinfo -o 524308480 -p 'PW' -f 5 -s /evidence/mbp-001.dd
Snapshot: 1
        Name   : com.apple.TimeMachine.2026-07-28-031500.local
        XID    : 198442
        Time   : Jul 28, 2026 03:15:02 UTC
Snapshot: 2
        Name   : com.apple.TimeMachine.2026-07-28-041502.local
        XID    : 199013

# apfs-fuse mounts one directly by name
└─╼ # apfs-fuse -v 4 -p 'PW' -s 'com.apple.TimeMachine.2026-07-28-031500.local' \
    -o ro ${LOOP}p2 /mnt/snap1
```

> ▲ DEVIL — SNAPSHOTS DEFEAT ANTI-FORENSICS FOR FREE
>
> An attacker deletes their toolkit and wipes the logs. If any local snapshot predates the cleanup — and one is created hourly on any Mac with Time Machine configured — **the deleted files and the original logs are sitting inside it, fully intact and trivially readable.** Snapshot enumeration should be one of the first things you do on any volume, not an afterthought. §42 is the full treatment, including how to detect snapshots that were themselves deleted.

15.4 Tombstones

Deleting a file removes its records from the current catalog tree, but snapshots referencing older XIDs still resolve them. The records are not gone; they are unreachable from the present. Walking older trees

directly — via snapshots (§42) or checkpoints (§41) — recovers them with full metadata, which is dramatically better than carving because you get the name, the path, and the timestamps back too.

16 Sealed System Volume and cryptexes

16.1 What the seal is

Since Big Sur the System volume is cryptographically sealed. A Merkle tree of SHA-256 hashes covers every file's contents; the tree's root hash — the **seal** — is stored in the volume's integrity metadata and verified at boot against Apple's signature. Any modification to any byte of the system volume changes the root hash and breaks the seal.

> ▸ **TIP — A BROKEN SEAL IS A HIGH-CONFIDENCE FINDING**
>
> You do not need Apple's expected value to make use of this. Recompute the tree from the volume's own `FILE_INFO` records and compare against the stored root hash. A mismatch means the system volume was modified after sealing — which, on a machine that boots normally, essentially requires deliberate action. §46 covers the procedure.

```
# confirm the volume is sealed and read the stored root hash
└─╼ $ fsapfsinfo -o 524308480 -f 1 /evidence/mbp-001.dd | grep -i -A4 'seal\|integrity'
        Compatible features       : sealed volume
        Integrity metadata:
                Hash type         : SHA-256
                Root hash         : 4f9c2a...e71b
                Flags             : 0x00000000
```

> ● **NOTE — THE "BROKEN SEAL" FLAG**
>
> The integrity metadata carries an `APFS_SEAL_BROKEN` flag which macOS itself sets when the seal is invalidated. If that flag is set, the system was booted with a knowingly modified system volume — which requires reduced security mode to have been enabled. That in turn leaves a record on the Preboot volume (§49). Two independent artifacts, one story.

16.2 Cryptexes 13+

Ventura introduced **cryptexes** — cryptographically sealed disk images mounted over the system at boot, used to ship Safari, dyld caches, and Rapid Security Responses without re-sealing the whole system volume. They live on the Preboot volume:

```
└─╼ $ ls /mnt/preboot/apfs2/Cryptexes/
App  OS  Incoming

└─╼ $ find /mnt/preboot/apfs2/Cryptexes -maxdepth 2 -name '*.dmg' -o -name '*.im4m' | head
```

Two reasons to look:

- **Version determination.** An RSR changes the effective OS build without changing `SystemVersion.plist`. If your artifact paths depend on the exact build — and some do — the cryptex tells you the truth.
- **An unusual persistence surface.** Cryptexes are signed and verified, so subverting one is not casual work. But their presence in `Incoming/`, unexpected contents, or a cryptex that does not correspond to a known Apple release deserves explanation.

V

THE TRADITIONAL WORKFLOW

The classical dead-box toolchain, applied to APFS. This is the part that produces the backbone of the examination: an inode-level inventory, a timeline, recovered deleted content, and a set of hash-filtered leads. Everything exotic later in this manual is an addition to this, never a replacement for it.

17 The Sleuth Kit run, start to finish
18 Timelines — body files, mactime, super-timelines
19 Carving and unallocated space
20 Plaso, log2timeline and Timesketch
21 Hashing, known-file filtering and YARA sweeps

17 The Sleuth Kit run

TSK addresses APFS through its pool abstraction. Every command needs two coordinates: the container's **sector offset** (`-o`, from `mmls`) and the volume's **APSB block** (`-B`, from `pstat`). Establish both once and reuse them.

```
# set up shell variables and never type the offsets again
└─╼ $ IMG=/evidence/mbp-001.dd
└─╼ $ OFF=1024040          # container start sector, from mmls
└─╼ $ SYS=1102             # System volume APSB block, from pstat
└─╼ $ DAT=1276             # Data volume APSB block
```

17.1 Volume statistics

```
└─╼ $ fsstat -o $OFF -B $DAT $IMG
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: APFS
Volume Name: Macintosh HD - Data
Volume UUID: 3E9A...
Capacity: 494384795648 bytes
Reserved: ...
Case Sensitive: No
Encrypted: Yes
Formatted By: hfs_convert (1412.101.1)
Created: 2023-09-14 11:02:41
Next Object ID: 2214877
Number of Files: 1874221
Number of Directories: 402118
```

Next Object ID is worth writing down. It is the high-water mark of inode allocation and the anchor for the anomaly detection in §43.

17.2 Listing the tree

```
# recursive listing with full paths; deleted entries are marked with *
└─╼ $ fls -o $OFF -B $DAT -r -p $IMG | head
d/d 2:  private
d/d 3:  Users
r/r 1024:       Users/jdoe/.zsh_history
-/r * 8871:     Users/jdoe/Downloads/installer.pkg      <-- deleted

# deleted entries only — the fastest lead generator on the whole image
└─╼ $ fls -o $OFF -B $DAT -r -p -d $IMG > ~/cases/001/06-analysis/deleted-entries.txt
└─╼ $ wc -l < ~/cases/001/06-analysis/deleted-entries.txt
3412
```

> ● **NOTE — DELETED-ENTRY RECOVERY IS WEAKER ON APFS**
>
> On NTFS or ext4, `fls -d` reliably finds unlinked entries because the metadata sits in a fixed table that is reused lazily. APFS removes records from a B-tree instead, so what `fls -d` surfaces is a much thinner and less predictable set. **Do not conclude a file was never present because `fls -d` did not list it.** The productive routes to deleted content on APFS are snapshots (§42) and checkpoints (§41), in that order.

17.3 Inode detail and extraction

```
# everything TSK knows about one inode, including the extent list
└──• $ istat -o $OFF -B $DAT $IMG 8871
Inode: 8871
Allocated: No
UID / GID: 501 / 20
Mode: rrw-r--r--
Size: 4823104
Created:  2026-07-21 19:04:11.492103000 (UTC)
Modified: 2026-07-21 19:04:16.881204000 (UTC)
Changed:  2026-07-30 02:41:55.003001000 (UTC)
Accessed: 2026-07-21 19:05:02.110448000 (UTC)
Direct Blocks:
884213 884214 884215 ...

# extract the content by inode
└──• $ icat -o $OFF -B $DAT $IMG 8871 > ~/cases/001/03-extract/recovered/installer.pkg
└──• $ file ~/cases/001/03-extract/recovered/installer.pkg
xar archive version 1

# bulk export of everything, allocated and unallocated
└──• $ tsk_recover -o $OFF -B $DAT -e $IMG ~/cases/001/03-extract/tsk-all/
```

▸ TIP — READ THAT istat OUTPUT AGAIN

Created, Modified and Accessed all cluster around 2026-07-21. **Changed** is nine days later, with the file already deleted. That is the timestomp signature from §14.1 appearing in real output. The metadata was altered long after the file was written — and someone bothered to do that to a file they then deleted.

18 Timelines

18.1 Body files and mactime

```
# generate a body file for each volume, tagging the source in the path
└─╼ $ fls -o $OFF -B $SYS -r -m "/sys" $IMG >  ~/cases/001/04-timeline/body.txt
└─╼ $ fls -o $OFF -B $DAT -r -m "/data" $IMG >> ~/cases/001/04-timeline/body.txt

# render to a timeline — always pin the timezone explicitly
└─╼ $ mactime -b ~/cases/001/04-timeline/body.txt -d -z UTC \
    > ~/cases/001/04-timeline/fs-timeline.csv

# narrow to a window of interest
└─╼ $ mactime -b ~/cases/001/04-timeline/body.txt -d -z UTC 2026-07-20..2026-08-01 \
    > ~/cases/001/04-timeline/fs-window.csv
```

■ CAUTION — ALWAYS `-z UTC`

`mactime` renders in the local timezone of the analysis box unless told otherwise. If your workstation drifts from UTC, or you move the case between machines, you will silently produce two timelines that disagree. Set the workstation to UTC (§3.4), pass `-z UTC` anyway, and state the timezone on every timeline exhibit.

18.2 Reading a body file directly

The format is pipe-delimited and trivially greppable, which is often faster than loading a timeline viewer:

```
MD5|name|inode|mode|UID|GID|size|atime|mtime|ctime|crtime
```

```
# everything created within a two-hour window, sorted
└─╼ $ awk -F'|' '$11 > 1753000000 && $11 < 1753007200 {print strftime("%F %T",$11), $2}' \
    ~/cases/001/04-timeline/body.txt | sort

# files whose ctime is more than 24h after their mtime — timestomp candidates
└─╼ $ awk -F'|' '$10 > $9 + 86400 {print strftime("%F %T",$9), strftime("%F %T",$10), $2}' \
    ~/cases/001/04-timeline/body.txt | sort | head -50
```

18.3 Super-timelines

A filesystem timeline alone is thin. The value comes from merging it with the artifact timelines from Part VII — unified logs, FSEvents, quarantine events, knowledgeC — into one ordered narrative. That merge is what plaso does (§20), and it is where an examination usually turns from a pile of facts into a sequence of events.

19 Carving and unallocated space

> **▲ DEVIL — CARVING IS THE WRONG FIRST INSTINCT ON APFS**
>
> Header/footer carving assumes files are contiguous. APFS, being copy-on-write with aggressive fragmentation, routinely scatters a file across dozens of extents in no useful order. Carvers will produce thousands of truncated, corrupt, or interleaved fragments and you will spend days triaging noise.
>
> Order of preference for recovering deleted content on APFS:
>
> 1. **Snapshots** (§42) — full files, full metadata, full paths
> 2. **Checkpoint reconstruction** (§41) — same, for states no snapshot covers
> 3. **Extent-aware recovery** — `tsk_recover`, `icat` by inode
> 4. **Carving** — last, and mainly for small self-contained formats: JPEG, PNG, PDF, SQLite pages

19.1 When you do carve

```
# PhotoRec is the most capable general carver; run it unattended against the image
└─╼ $ photorec /d ~/cases/001/05-carve/photorec /cmd $IMG partition_none,fileopt,everything,enable,search

# foremost for a targeted set — fast and predictable
└─╼ $ foremost -t jpg,png,pdf,doc,zip,sqlite -i $IMG -o ~/cases/001/05-carve/foremost

# scalpel when you need custom signatures (edit /etc/scalpel/scalpel.conf first)
└─╼ $ scalpel -c /etc/scalpel/scalpel.conf -o ~/cases/001/05-carve/scalpel $IMG
```

19.2 bulk_extractor — the one that earns its runtime

Unlike a carver, `bulk_extractor` ignores filesystem structure entirely and scans for features: email addresses, URLs, domains, credit-card numbers, phone numbers, JSON, PII, and crucially **it decompresses and recurses**. It finds content inside compressed streams, inside swap, inside unallocated space, and inside fragments no carver would reassemble.

```
└─╼ $ bulk_extractor -o ~/cases/001/05-carve/bulk -e all -S ssn_mode=1 $IMG

# the outputs worth reading first
└─╼ $ ls ~/cases/001/05-carve/bulk/
url.txt  url_histogram.txt  email.txt  email_histogram.txt  domain.txt
telephone.txt  ccn.txt  json.txt  windirs.txt  zip.txt  report.xml

# the histograms are the point — ranked, deduplicated, immediately readable
└─╼ $ head -30 ~/cases/001/05-carve/bulk/url_histogram.txt
```

> **▸ TIP — RUN IT AGAINST THE ENCRYPTED IMAGE TOO**
>
> Run `bulk_extractor` against the raw image as well as the decrypted mount. The raw image includes the unencrypted volumes — Preboot, Recovery, VM — and any region the FUSE mount does not expose. It costs one long unattended run and occasionally surfaces material nothing else does.

19.3 The unallocated-space problem

```
# the classical approach — which you should expect to work poorly here
└─╼ $ blkls -o $OFF -B $DAT $IMG > ~/cases/001/05-carve/unalloc.raw
```

TSK's notion of "unallocated" maps badly onto a copy-on-write filesystem with a space-manager bitmap, and against an encrypted volume it cannot work at all. The reliable route is to read the space manager's allocation bitmap directly and extract the unreferenced blocks yourself — §45.

20 Plaso, log2timeline and Timesketch

Plaso parses filesystem metadata and several hundred artifact formats into one normalised timeline. On APFS it reaches the filesystem through dfVFS on top of libfsapfs — the library you built in §7.3.

```
└─╼ $ python3 -m pip install --user plaso

# full run over the image; supply the FileVault credential so it can read Data
└─╼ $ log2timeline.py --storage_file ~/cases/001/04-timeline/plaso.db \
    --partitions all --volumes all \
    --credential password:'TheUserPassword' \
    --hashers sha256 \
    $IMG

# filter to a window and export
└─╼ $ psort.py -o dynamic -w ~/cases/001/04-timeline/super-timeline.csv \
    ~/cases/001/04-timeline/plaso.db \
    "date > '2026-07-20 00:00:00' AND date < '2026-08-01 00:00:00'"

# what did it actually parse? always check before trusting completeness
└─╼ $ pinfo.py ~/cases/001/04-timeline/plaso.db | head -60
```

■ CAUTION — CONFIRM THE CREDENTIAL SYNTAX AND THAT IT WORKED

Plaso's credential flag has changed form across releases. Run `log2timeline.py --help | grep -A3 credential` against your installed version rather than copying syntax from any guide, this one included. Then verify positively with `pinfo.py` that events were extracted from the Data volume. A run that silently failed to decrypt still produces a plausible-looking storage file — populated entirely from the unencrypted volumes.

▸ TIP — TIMESKETCH FOR ANYTHING NON-TRIVIAL

A CSV of four million timeline events is not analysable in a spreadsheet. Timesketch gives you search, tagging, saved views, and multiple timelines overlaid on one axis — which is exactly the shape of the correlation work in §53. It runs in Docker on the Parrot box.

```
└─╼ $ git clone https://github.com/google/timesketch.git ~/tools/timesketch
└─╼ $ cd ~/tools/timesketch && docker compose up -d
└─╼ $ psort.py -o timesketch --name "case-001-fs" ~/cases/001/04-timeline/plaso.db
```

21 Hashing, known-file filtering and YARA

21.1 Reduce the haystack

A macOS install contains well over a million files, almost all of them Apple's. Filtering known-good content is the difference between a tractable examination and an impossible one.

```
# hash everything on the mounted volumes
└─╼ $ hashdeep -c sha256 -r /mnt/data/apfs5 > ~/cases/001/01-image/data-volume.hashes

# build a TSK hash database from an NSRL set and index it
└─╼ $ hfind -i nsrl-sha1 /opt/nsrl/NSRLFile.txt
└─╼ $ hfind /opt/nsrl/NSRLFile.txt 8a5f...c21b
8a5f...c21b     Microsoft Word
```

▲ DEVIL — THE SEALED VOLUME IS ITS OWN HASH SET

You do not need NSRL to filter Apple's own files. The sealed System volume carries a `FILE_INFO` record — a SHA-256 — for **every file it contains**, placed there by Apple. That is a complete, authoritative, locally available known-good set for the entire OS, already on your image. Extract those hashes (§46) and use them as your filter. It is more accurate than any external hash set, because it is version-exact for this specific machine.

21.2 YARA across the mount

```
└─╼ # apt install -y yara
└─╼ $ git clone https://github.com/Yara-Rules/rules.git ~/tools/yara-rules

# scan the user data, following nothing, reporting matched strings
└─╼ $ yara -r -s -w ~/tools/yara-rules/index.yar /mnt/data/apfs5/Users/ \
    2> /dev/null | tee ~/cases/001/06-analysis/yara-users.txt

# macOS-specific rule sets are far higher signal than generic ones
└─╼ $ git clone https://github.com/objective-see/Malware.git ~/tools/objsee-samples
```

▸ TIP — SCAN THE SNAPSHOTS, NOT JUST THE PRESENT

Mount each snapshot (§42) and run the same YARA sweep across it. Malware deleted before acquisition is frequently still present, intact, inside a snapshot from a few hours earlier — and a YARA hit inside a snapshot is a far stronger finding than a carved fragment, because it comes with a path, a timestamp and an inode.

VI

PARROT'S ARSENAL

What is already on the box, what has to be added, and which tool to reach for when. Parrot ships a strong general forensics stack; what it does not ship — and cannot, because Debian does not package them — is everything macOS-specific. This is the gap list.

22 What ships in Parrot, what you must add

23 The toolset, category by category

22 What ships, what you must add

Parrot's Security and Forensics editions carry most of the classical stack in the default metapackages. Confirm what you actually have rather than assuming:

```
# what forensics packages are installed right now
└─• $ apt list --installed 2>/dev/null | grep -Ei \
    'sleuth|tsk|ewf|aff|foremost|scalpel|photorec|testdisk|bulk|binwalk|yara|volatility|hashdeep'

# Parrot's forensics metapackage, if you are on a minimal install
└─• # apt install -y parrot-tools-forensics
```

Area	Ships with Parrot	Must be added
Imaging	dd, dc3dd, dcfldd, ddrescue, guymager, ewf-tools, afflib-tools	—
APFS access	~~nothing~~	apfs-fuse, libfsapfs, linux-apfs-oot, apfsprogs
Filesystem analysis	sleuthkit, autopsy	Newer TSK from source if pstat is missing
Carving	foremost, scalpel, testdisk/photorec, magicrescue, binwalk, bulk-extractor	—
Timeline	mactime (with TSK)	plaso, timesketch
macOS artifacts	~~nothing~~	mac_apt, APOLLO, unified-log parsers, FSEventsParser, spotlight_parser, chainbreaker
SQLite	sqlite3, sqlitebrowser	undark, walitean, sqlparse, bring2lite
Plists	libplist-utils (plistutil)	ccl_bplist, nska_deserialize
Malware triage	yara, radare2, clamav, exiftool	LIEF, capa, ghidra
Memory	volatility3 (edition-dependent)	macOS symbol tables — and see §59
Network	wireshark, tcpflow, tshark, networkminer	—

> ● **NOTE — THE PATTERN**
>
> Everything generic is packaged; everything macOS-specific is not. That is not an oversight in Parrot — the macOS DFIR ecosystem is largely Python on GitHub, moves fast, and is not packaged anywhere. Plan on a `~/tools` directory of git clones and a Python virtualenv, and plan on recording commit hashes (§5.4).

23 The toolset, category by category

23.1 Set up an isolated Python environment first

Most of what follows is Python, some of it with conflicting dependencies. Do not install it into the system interpreter on a machine you rely on.

```
└─╼ $ python3 -m venv ~/tools/venv-dfir
└─╼ $ source ~/tools/venv-dfir/bin/activate
└─╼ $ pip install --upgrade pip wheel

# the core set — details on each below
└─╼ $ pip install plaso pytsk3 pyewf construct lief capstone \
    biplist ccl-bplist nska-deserialize pandas python-dateutil
```

23.2 The macOS artifact parsers

Tool	Source	What it does
mac_apt	ydkhatri/mac_apt	The closest thing to an all-in-one macOS artifact suite. Runs on Linux, reads images directly, ~40 plugins covering most of Part VII. Start here. §60
macos-UnifiedLogs	mandiant/macos-UnifiedLogs	Rust parser for .tracev3. The only reliable way to read unified logs off-Mac. §24
UnifiedLogReader	ydkhatri/UnifiedLogReader	Python alternative; slower, sometimes handles files the Rust parser rejects
FSEventsParser	dlcowen/FSEventsParser	Decodes /.fseventsd gzipped event streams. §25
spotlight_parser	ydkhatri/spotlight_parser	Parses store.db metadata — filenames and attributes for files long deleted. §26
APOLLO	mac4n6/APOLLO	Pattern-of-life correlation over knowledgeC, powerlog, CoreDuet and friends. §61
chainbreaker	n0fate/chainbreaker	Keychain extraction. §56
nska_deserialize	PyPI	Turns NSKeyedArchiver plists into readable structures — needed constantly. §33

```
# clone the parser set in one go
└─╼ $ mkdir -p ~/tools && cd ~/tools
└─╼ $ for r in ydkhatri/mac_apt mandiant/macos-UnifiedLogs dlcowen/FSEventsParser \
        ydkhatri/spotlight_parser mac4n6/APOLLO n0fate/chainbreaker; do
    git clone --depth 1 "https://github.com/$r.git"
  done

# the Mandiant parser is Rust — build it once
└─╼ $ cd ~/tools/macos-UnifiedLogs && cargo build --release --examples
```

23.3 SQLite forensics tools

Tool	Use
sqlite3	Queries, plus .recover which rebuilds from damaged or partially-freed pages
undark	Extracts deleted rows from freelist and unallocated page space
walitean	Recovers records from -wal write-ahead logs — frequently holds the newest, uncommitted activity
sqlparse (mdegrazia)	Parses journal and WAL files into readable records
bring2lite	Research-grade deleted-record recovery; slow, thorough
sqlitebrowser	Fast interactive exploration when you do not yet know the schema

23.4 Binary and Mach-O triage

Tool	Use on Mach-O without Apple tooling
LIEF	Python: parse Mach-O headers, load commands, entitlements, embedded code-signature blobs. The workhorse. §57
radare2 / rizin	Disassembly, strings, cross-references, symbol listing
ghidra	Full decompilation when triage is not enough
capa	Maps binary capabilities to ATT&CK — fast triage verdicts
exiftool	Metadata from documents and images; reads bundle plists too
ent / binwalk -E	Entropy measurement — packed and encrypted content. §58

VII

THE ARTIFACT HUNT

macOS records an extraordinary amount about itself, in a dozen formats, in places chosen apparently at random. This part is the map. Every path is given as macOS sees it and as it appears on your Linux mount, because on Ventura those are never the same thing.

24 The unified log — .tracev3 and uuidtext
25 FSEvents — the filesystem's own diary
26 Spotlight metadata stores
27 Quarantine and download provenance
28 Pattern of life — knowledgeC, powerlog, CoreDuet
29 TCC — the permission ledger
30 Notifications, installs and BSM audit logs
31 Browsers, Messages, Mail and user data
32 SQLite deep dive — WAL, freelist, unallocated
33 Plists and NSKeyedArchiver without plutil

24 The unified log

Since Sierra, almost everything the system says about itself goes into the unified log: process launches, network state, authentication, mounts, USB attachment, sandbox violations, TCC decisions. It is the richest single artifact on the machine and the most annoying to read off-Mac.

Log data	`/private/var/db/diagnostics/` → `<DATA>/private/var/db/diagnostics/`
String tables	`/private/var/db/uuidtext/` → `<DATA>/private/var/db/uuidtext/`

```
diagnostics/
├── Persist/        0000000000000001.tracev3   long-lived records — your main source
├── Special/        short-lived, higher-volume records
├── Signpost/       performance instrumentation
├── HighVolume/     very chatty subsystems
├── logdata.LiveData.tracev3    the in-flight buffer at shutdown/acquisition
└── Persist/...
uuidtext/
├── XX/YYYYYYYY...  per-binary format-string tables
└── dsc/            shared cache string tables
```

▲ DEVIL — THE LOG IS USELESS WITHOUT uuidtext

`.tracev3` files do not contain log messages. They contain format string references and arguments; the actual text lives in `uuidtext/`, keyed by the UUID of the binary that emitted it. Copy only `diagnostics/` and you get a parse full of `<missing format string>`. **Always extract both directories together**, preserving their relative layout. This single mistake accounts for most "the unified log parser doesn't work" complaints.

24.1 Extract and parse

```
# extract both trees, preserving structure and timestamps
└─╼ $ mkdir -p ~/cases/001/03-extract/logs
└─╼ $ cp -a /mnt/data/apfs5/private/var/db/diagnostics ~/cases/001/03-extract/logs/
└─╼ $ cp -a /mnt/data/apfs5/private/var/db/uuidtext    ~/cases/001/03-extract/logs/

# Mandiant's Rust parser — fastest and most complete
└─╼ $ ~/tools/macos-UnifiedLogs/target/release/examples/unifiedlog_parser \
    -i ~/cases/001/03-extract/logs -o ~/cases/001/04-timeline/unifiedlog.csv

└─╼ $ wc -l ~/cases/001/04-timeline/unifiedlog.csv
14882301 unifiedlog.csv
```

24.2 What to look for in fifteen million lines

Signal	Grep for
Process execution	`xpcproxy`, `execve`, `launchd.*spawn`, `Sandbox`
launchd job loads	`com.apple.xpc.launchd`, `service state`, the label of the suspect job
USB / external media	`IOUSBHostDevice`, `USBMSC`, `diskarbitrationd`, `mount`
Authentication	`opendirectoryd`, `authorization`, `SecurityAgent`, `sudo`
TCC decisions	`tccd`, `TCCAccessRequest`, `kTCCService`
Gatekeeper / notarisation	`syspolicyd`, `XProtect`, `notarization`, `assessment`
Network changes	`configd`, `SCNetwork`, `en0`, VPN interface names
Screen sharing / remote	`screensharingd`, `ARDAgent`, `sshd`
Clock changes	`settimeofday`, `timed`, `ntpd` — see §53

```
# process launches in a window, as a first pass
└──• $ awk -F',' '$1 > "2026-07-30 00:00" && $1 < "2026-07-31 00:00"' \
    ~/cases/001/04-timeline/unifiedlog.csv | grep -i 'xpcproxy\|execve' | head -50

# every time the system clock was set — anti-forensics indicator
└──• $ grep -iE 'settimeofday|time (change|jump)|adjusting local clock' \
    ~/cases/001/04-timeline/unifiedlog.csv
```

▲ DEVIL — GAPS ARE THE FINDING

Unified log files rotate and are pruned automatically, so absence alone proves nothing. But `.tracev3` files carry sequential identifiers, and the record stream is continuous. A **missing file in the sequence**, or a clean multi-hour hole in an otherwise chatty log with no corresponding shutdown record, is not normal pruning. §54 covers how to distinguish rotation from deletion.

25 FSEvents

Every APFS volume keeps a low-level record of filesystem changes in `/.fseventsd/` at its root. It is coarse — it records that a path changed and roughly how, not who or exactly when — but it reaches back weeks or months and it records **paths that no longer exist**.

Location	`<DATA>/.fseventsd/` and on every other volume too
Format	gzip-compressed binary streams, named with hex event IDs; plus `fseventsd-uuid`

```
└─╼ $ ls /mnt/data/apfs5/.fseventsd/ | head
0000000000a1b2c3
0000000000a1b2d4
fseventsd-uuid

└─╼ $ cp -a /mnt/data/apfs5/.fseventsd ~/cases/001/03-extract/logs/

└─╼ $ python3 ~/tools/FSEventsParser/FSEParser_V4.1.py \
    -c case001 -s ~/cases/001/03-extract/logs/.fseventsd \
    -o ~/cases/001/04-timeline/fsevents/

# the output is per-path with change-type flags
└─╼ $ grep -i 'Downloads' ~/cases/001/04-timeline/fsevents/*.tsv | head
```

▲ DEVIL — FSEVENTS REMEMBERS DELETED FILES BY NAME

An FSEvents record persists long after the file it describes is gone. You will routinely find entries for files that exist nowhere on the volume, in no snapshot, and in no carved output — proving that a file with that exact path existed and was created, modified, renamed or removed. For establishing that a tool was present on the machine before being wiped, this is often the only surviving evidence. It carries no content, but "a file named `Users/jdoe/Library/LaunchAgents/com.apple.softwareupdated.plist` was created and later removed" is frequently the whole point.

Check `/.fseventsd` on **every** volume, not just Data. And note that the presence of a `fseventsd-uuid` that differs from the one in your snapshots means the volume's event history was reset — which happens on erase, and on deliberate deletion of the directory.

26 Spotlight metadata stores

Spotlight indexes far more than filenames — it stores per-file metadata attributes, including where a file was downloaded from, its content type, its title, and in many cases a text preview. Crucially, index entries can survive the file itself.

Volume store	`<DATA>/.Spotlight-V100/Store-V2/<UUID>/store.db` and `.store.db`
Also present on	every mounted volume, including external drives that were attached
User store	`<DATA>/private/var/folders/<xx>/<yyy>/0/com.apple.Spotlight/`

```
└─╴ $ find /mnt/data/apfs5/.Spotlight-V100 -name 'store.db' -o -name '.store.db'

└─╴ $ python3 ~/tools/spotlight_parser/spotlight_parser.py \
    /mnt/data/apfs5/.Spotlight-V100/Store-V2/3E9A.../store.db \
    ~/cases/001/06-analysis/spotlight/

# the attribute that matters most: download provenance, retained per file
└─╴ $ grep -i 'kMDItemWhereFroms\|kMDItemDownloadedDate' \
    ~/cases/001/06-analysis/spotlight/*.txt | head -40
```

▸ TIP — EXTERNAL DRIVE STORES ARE A HISTORY OF OTHER MACHINES

If any external volume was indexed on this Mac, its `.Spotlight-V100` travelled with it. Conversely, if the subject attached a drive that had been indexed elsewhere, its store may describe files from a completely different computer. Always check which volume UUID a store belongs to before attributing its contents to this machine.

27 Quarantine and download provenance

Three independent records of where a file came from. They disagree often enough that you should collect all three.

27.1 The quarantine database

```
# ~/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2
└─► $ Q=/mnt/data/apfs5/Users/jdoe/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2
└─► $ cp -a "$Q"* ~/cases/001/03-extract/db/

└─► $ sqlite3 ~/cases/001/03-extract/db/com.apple.LaunchServices.QuarantineEventsV2 \
  "SELECT datetime(LSQuarantineTimeStamp + 978307200, 'unixepoch') AS ts,
          LSQuarantineAgentName, LSQuarantineDataURLString, LSQuarantineOriginURLString
   FROM LSQuarantineEvent ORDER BY ts DESC LIMIT 40;"
2026-07-21 19:03:58|Safari|https://cdn.example.net/installer.pkg|https://example.net/download
```

● NOTE — THE 978307200 CONSTANT

Apple timestamps in these databases are **Mac absolute time**: seconds since 2001-01-01 UTC. Add `978307200` to convert to Unix epoch. You will type this number constantly — see Appendix C. Getting it wrong shifts every timestamp by 31 years, which is at least obvious. Forgetting that some tables use Unix epoch and others use Mac absolute in the same database is the subtler trap.

27.2 The quarantine extended attribute

```
# the xattr rides on the file itself and survives copying within the volume
└─► $ getfattr -n com.apple.quarantine -d /mnt/data/apfs5/Users/jdoe/Downloads/installer.pkg
com.apple.quarantine="0083;68d3f1a6;Safari;F1E2D3C4-..."
#  flags ; hex mac-absolute timestamp ; agent ; event UUID (joins to the DB above)

# sweep the whole user tree for quarantined files, including ones the DB forgot
└─► $ getfattr -R -n com.apple.quarantine /mnt/data/apfs5/Users/ 2>/dev/null | \
    grep -B1 'com.apple.quarantine' | head -60
```

27.3 The where-from attribute

```
# com.apple.metadata:kMDItemWhereFroms is a binary plist inside an xattr
└─► $ getfattr -n com.apple.metadata:kMDItemWhereFroms --only-values \
    /mnt/data/apfs5/Users/jdoe/Downloads/installer.pkg > /tmp/wf.bplist
└─► $ plistutil -i /tmp/wf.bplist -o -
<array>
  <string>https://cdn.example.net/installer.pkg</string>
  <string>https://example.net/download</string>
</array>
```

▲ DEVIL — ABSENCE OF QUARANTINE IS ITSELF A SIGNAL

Anything downloaded by a normal browser gets quarantined. A recently-created executable or application bundle in a user directory with **no** quarantine xattr arrived by a route that does not set it: `curl`, `wget`, `scp`, AirDrop in some configurations, a package installer running as root, extraction from an archive by a non-quarantine-aware tool, or deliberate removal with `xattr -d`. On a machine whose user only ever installs things from the browser, an unquarantined binary in `~/Downloads` is worth more attention than a quarantined one.

28 Pattern of life — knowledgeC, powerlog, CoreDuet

Apple's on-device analytics record application usage, focus, device state and activity at a granularity that surprises people. These databases answer "what was the user actually doing at 14:32" better than anything else on the machine.

Artifact	Path (Data-volume relative)
knowledgeC (system)	private/var/db/CoreDuet/Knowledge/knowledgeC.db
knowledgeC (user)	Users/<u>/Library/Application Support/Knowledge/knowledgeC.db
interactionC	private/var/db/CoreDuet/People/interactionC.db
Powerlog	private/var/db/powerlog/Library/BatteryLife/CurrentPowerlog.PLSQL
Powerlog archives	private/var/db/powerlog/Library/BatteryLife/Archives/
Screen Time	Users/<u>/Library/Application Support/Knowledge/

```
# application usage with durations, from knowledgeC
└──╼ $ sqlite3 knowledgeC.db "
SELECT datetime(ZOBJECT.ZSTARTDATE + 978307200,'unixepoch') AS start,
       datetime(ZOBJECT.ZENDDATE   + 978307200,'unixepoch') AS end,
       ZOBJECT.ZVALUESTRING AS app,
       (ZOBJECT.ZENDDATE - ZOBJECT.ZSTARTDATE) AS seconds
FROM ZOBJECT
WHERE ZSTREAMNAME = '/app/inFocus'
ORDER BY start DESC LIMIT 50;"
2026-07-30 02:38:11|2026-07-30 02:44:02|com.apple.Terminal|351
```

▲ DEVIL — POWERLOG OUTLIVES THE APPLICATION

Powerlog records per-process energy and activity for weeks, and **it keeps recording bundle identifiers for applications that have since been deleted**. An application installed, run once and removed leaves no bundle, no Launch Services entry, and often no log — but it leaves a powerlog row. Combined with knowledgeC's focus intervals you can reconstruct that an unknown binary ran, for how long, and when, from an artifact almost nobody wipes.

```
└──╼ $ sqlite3 CurrentPowerlog.PLSQL ".tables" | tr ' ' '\n' | grep -i 'app\|process' | head
```

▸ TIP — USE APOLLO RATHER THAN HAND-WRITTEN SQL

APOLLO ships hundreds of vetted queries across these databases and normalises the output into one timeline. Writing your own SQL is for when you need something APOLLO does not have. §61.

29 TCC — the permission ledger

Transparency, Consent and Control records every privacy permission ever granted or denied: camera, microphone, screen recording, full disk access, accessibility, automation. For an intrusion case it is close to a list of what the attacker wanted.

System	`<DATA>/Library/Application Support/com.apple.TCC/TCC.db`
Per user	`<DATA>/Users/<u>/Library/Application Support/com.apple.TCC/TCC.db`

```
└──╼ $ sqlite3 TCC.db "
SELECT service, client, client_type, auth_value, auth_reason,
       datetime(last_modified,'unixepoch') AS modified
FROM access ORDER BY last_modified DESC;"
kTCCServiceScreenCapture|com.example.updater|0|2|3|2026-07-30 02:39:44
kTCCServiceAccessibility|com.example.updater|0|2|3|2026-07-30 02:39:51
```

Service	Why an attacker wants it
kTCCServiceAccessibility	Synthetic input and full UI control — the single most abused permission on macOS
kTCCServiceScreenCapture	Screen recording
kTCCServiceSystemPolicyAllFiles	Full Disk Access — reads every user file including other apps' databases
kTCCServiceMicrophone / Camera	Surveillance
kTCCServiceAppleEvents	Driving other applications via automation
kTCCServiceSystemPolicyDesktopFolder etc	Targeted access to Desktop, Documents, Downloads

▲ DEVIL — TCC IS A TIMELINE OF INTENT

`last_modified` tells you when each grant happened, and grants cluster: a user installing a legitimate screen-sharing tool produces two or three related entries within a minute. An unfamiliar bundle identifier acquiring Accessibility and Screen Recording and Full Disk Access inside the same thirty seconds is a compromise timestamp accurate to the second.

Cross-reference every `client` value against the bundles actually present on disk. A TCC entry naming a bundle that no longer exists means the application was removed after being granted access — and the entry survived the cleanup.

30 Notifications, installs and audit logs

30.1 Notification Center

Holds delivered notification content — message previews, mail subjects, alerts — often including text from applications whose own databases were cleared.

```
└─╼ $ find /mnt/data/apfs5/private/var/folders -path '*com.apple.notificationcenter/db2/db' 2>/dev/null
└─╼ $ sqlite3 db "SELECT datetime(delivered_date+978307200,'unixepoch'), app_id, data
            FROM record ORDER BY delivered_date DESC LIMIT 30;"
# the 'data' column is an NSKeyedArchiver blob — see §33
```

30.2 install.log

Plain text, long-retained, and it records every package installation, OS update and — importantly — every `installer` invocation regardless of source.

```
└─╼ $ grep -iE 'installer|package|Install(ed|ing)' \
    /mnt/data/apfs5/private/var/log/install.log | tail -100

# rotated copies go back much further than people expect
└─╼ $ ls -la /mnt/data/apfs5/private/var/log/install.log*
└─╼ $ zgrep -i 'pkg' /mnt/data/apfs5/private/var/log/install.log.*.gz | tail -50
```

30.3 OpenBSM audit logs

The BSM audit trail is the closest thing macOS has to Linux auditd. It is **off by default on recent macOS**, but where it was enabled — managed fleets, hardened builds — it is authoritative process-execution evidence.

```
└─╼ $ ls /mnt/data/apfs5/private/var/audit/ | tail
20260730024155.20260730031200
20260730031200.not_terminated

# Debian has no praudit. Parse with mac_apt's plugin, or with Python:
└─╼ $ python3 ~/tools/mac_apt/mac_apt.py -o ~/cases/001/06-analysis/macapt \
    MOUNTED /mnt/data/apfs5 BASICINFO
```

> ● **NOTE — CHECK WHETHER AUDITING WAS EVEN ON**
>
> Read `<DATA>/private/etc/security/audit_control` before drawing conclusions from an empty `/var/audit`. If auditing was disabled the absence means nothing. If it was enabled and the directory is empty or truncated, that is a different and much more interesting fact.

31 Browsers, Messages, Mail and user data

Artifact	Path (Data-volume relative)	Notes
Safari history	Users/<u>/Library/Safari/History.db	Mac absolute time; check -wal
Safari downloads	Users/<u>/Library/Safari/Downloads.plist	bplist; includes in-progress and removed items
Safari session	Users/<u>/Library/Safari/LastSession.plist	Tabs open at last quit — often very revealing
Safari (sandboxed)	Users/<u>/Library/Containers/com.apple.Safari/Data/Library/Safari/	13+ some files moved into the container
Chrome	Users/<u>/Library/Application Support/Google/Chrome/Default/History	WebKit epoch (1601, µs)
Firefox	Users/<u>/Library/Application Support/Firefox/Profiles/*/places.sqlite	Unix µs
Messages	Users/<u>/Library/Messages/chat.db	Plus Attachments/ and a large -wal
Mail	Users/<u>/Library/Mail/V10/	13+ V10; Envelope Index is SQLite
Photos	Users/<u>/Pictures/Photos Library.photoslibrary/database/Photos.sqlite	Retains records for deleted assets
Recent items	Users/<u>/Library/Application Support/com.apple.sharedfilelist/*.sfl2	NSKeyedArchiver; recent docs, apps, servers
Shell history	Users/<u>/.zsh_history, .bash_history	First thing to read on any intrusion case
SSH	Users/<u>/.ssh/known_hosts, config, authorized_keys	Where they went, and who can get in
Folder state	.DS_Store throughout	Records filenames that were once in a folder

▲ DEVIL — .DS_Store REMEMBERS WHAT WAS IN THE FOLDER

`.DS_Store` stores per-folder Finder state keyed by filename. Delete a file and its entry frequently remains. A `.DS_Store` in a directory that is now empty can enumerate exactly what used to be there, complete with names. Parse them with `python3 -m ds_store` or `mac_apt`, and do it across the whole volume rather than one directory at a time.

32 SQLite deep dive

Nearly every artifact above is SQLite. Treating those files as "run a SELECT and move on" throws away most of the available evidence.

32.1 Always take the whole set

```
# NEVER copy just the .db
└─╼ $ cp -a chat.db chat.db-wal chat.db-shm ~/cases/001/03-extract/db/
```

▲ DEVIL — THE NEWEST DATA IS IN THE -wal, NOT THE DATABASE

In WAL mode — which macOS uses almost everywhere — committed transactions live in the `-wal` file until a checkpoint folds them into the main database. On a machine that was imaged without a clean shutdown, **the most recent hours of activity exist only in the `-wal`**. Copy the `.db` alone and you silently lose the very period you most care about.

Worse: opening the database with `sqlite3` while the `-wal` is present causes an automatic checkpoint, modifying your copy. Work on copies, and hash before and after.

32.2 Reading the WAL separately

```
# see what the WAL holds before it is merged
└─╼ $ python3 ~/tools/walitean/walitean.py -f chat.db-wal -o ~/cases/001/06-analysis/wal/

# or force a read-only open that will not checkpoint
└─╼ $ sqlite3 "file:chat.db?immutable=1" "PRAGMA journal_mode; SELECT count(*) FROM message;"
```

32.3 Deleted rows

```
# sqlite's own recovery — rebuilds from pages, including some freed content
└─╼ $ sqlite3 chat.db ".recover" > recovered.sql
└─╼ $ sqlite3 recovered.db < recovered.sql

# undark walks freelist and unallocated page space for deleted records
└─╼ $ undark -i chat.db --freespace > deleted-rows.txt

# crude but effective: strings across the raw database file
└─╼ $ strings -a -t d chat.db | grep -i 'suspicious-term'
```

▸ TIP — CHECK auto_vacuum BEFORE YOU INVEST TIME

`PRAGMA auto_vacuum;` returning `1` (FULL) means freed pages are returned to the OS immediately and deleted-row recovery will yield little. Returning `0` (NONE) means freed pages linger and recovery is worth the effort. One query saves an afternoon.

33 Plists and NSKeyedArchiver without plutil

Property lists come in three flavours: XML, binary (bplist), and binary containing an NSKeyedArchiver object graph. The third looks like nonsense until deserialised, and a great deal of modern macOS state is stored that way.

33.1 XML and binary plists

```
# Debian's libplist-utils replaces plutil for straightforward cases
└─╼ $ plistutil -i com.apple.dock.plist -o - | head -40

# Python handles both formats natively and is easier to script
└─╼ $ python3 -c "
import plistlib,sys,json
d=plistlib.load(open(sys.argv[1],'rb'))
print(json.dumps(d,indent=2,default=str))" com.apple.dock.plist
```

33.2 NSKeyedArchiver

Recognise it by a top-level `$archiver`, `$objects`, `$top` structure. `plistlib` will parse it and hand you an unusable flat object table; you need a deserialiser to rebuild the graph.

```
└─╼ $ pip install nska-deserialize
└─╼ $ python3 - <<'PY'
import nska_deserialize as nd, sys, json
with open('/path/to/com.apple.LSSharedFileList.RecentDocuments.sfl2','rb') as f:
    obj = nd.deserialize_plist(f)
print(json.dumps(obj, indent=2, default=str))
PY
```

Artifact	Why it is NSKeyedArchiver
`*.sfl2` shared file lists	Recent documents, applications, servers, mounted volumes
`BackgroundItems-v*.btm`	The modern login-item store — §35
Notification Center `record.data`	Notification content
`com.apple.finder.plist` recents	Finder state
Many `Library/Preferences` values	Anything storing a custom object

> ■ **CAUTION — PLIST TIMESTAMPS ARE MAC ABSOLUTE**
>
> `plistlib` returns real `datetime` objects for XML plist dates and handles them correctly. But a raw numeric timestamp stored *inside* a plist value — which is how most Apple code stores time — is almost always Mac absolute (2001 epoch) and needs `+978307200`. If a date comes out in 1988, you forgot. If it comes out in 2058, you added it twice.

VIII

THE PERSISTENCE MAP

Everywhere on Ventura and later that code can arrange to run again. Ordered roughly by how often it is actually used, not by how exotic it is — because the boring locations are where the findings usually are, and the obscure ones are where a competent adversary goes when they expect you to look at the boring ones.

34 launchd — agents and daemons
35 BTM — the modern login-item store
36 Scheduled execution — cron, at, periodic
37 Profiles, MDM and system extensions
38 Plugin and injection surfaces
39 Unix-side persistence
40 The persistence sweep — one-pass checklist

34 launchd — agents and daemons

Still the overwhelming majority of real-world macOS persistence. Six directories, and you must check all six because they differ in privilege and in which volume they live on.

Path on your mount	Runs as	Notes
<DATA>/Library/LaunchDaemons/	root, at boot	**Highest value.** Writable, survives logout, full privilege
<DATA>/Library/LaunchAgents/	each user, at login	Writable, applies to every user on the machine
<DATA>/Users/<u>/Library/LaunchAgents/	that user	**No admin rights needed to create.** Most common foothold
<SYS>/System/Library/LaunchDaemons/	root	Sealed — a modification here breaks the seal (§46)
<SYS>/System/Library/LaunchAgents/	each user	Sealed, same reasoning
<DATA>/Library/StartupItems/	root	Long deprecated; its presence at all is anomalous

34.1 Sweep and triage

```
# collect every job definition with its path, sorted by modification time
└─➤ $ find /mnt/data/apfs5/Library/Launch{Daemons,Agents} \
     /mnt/data/apfs5/Users/*/Library/LaunchAgents \
     /mnt/sys/apfs1/System/Library/Launch{Daemons,Agents} \
     -name '*.plist' -printf '%T@ %p\n' 2>/dev/null | sort -rn | head -40

# dump the interesting keys from each
└─➤ $ for p in /mnt/data/apfs5/Library/LaunchDaemons/*.plist; do
    echo "=== $p"
    plistutil -i "$p" -o - | grep -A3 -E 'Label|Program|RunAtLoad|KeepAlive|StartInterval|WatchPaths|DYLD'
  done
```

34.2 What makes a job suspicious

Key	Read it as
Label	Apple-lookalike names are the classic tell: com.apple.softwareupdated in ~/Library/LaunchAgents is not Apple's — Apple's live on the sealed volume
Program / ProgramArguments	The payload. Interpreters (osascript, python3, bash -c, curl | sh) and paths in /tmp, /usr/local, ~/Library are all elevated interest
RunAtLoad + KeepAlive	Start immediately and restart forever — the persistence idiom
StartInterval / StartCalendarInterval	Beaconing. A short interval on a non-Apple job is a C2 heartbeat until shown otherwise
WatchPaths / QueueDirectories	Trigger on filesystem activity — used to re-establish persistence when a file is deleted
StartOnMount	Fires whenever any volume mounts — used to target removable media
EnvironmentVariables	DYLD_INSERT_LIBRARIES here is library injection into whatever the job runs
LimitLoadToSessionType	Aqua = GUI login only; Background/LoginWindow = runs pre-login

▲ DEVIL — THE PLIST WITHOUT A BINARY, AND THE BINARY WITHOUT A PLIST

Two asymmetric checks that catch cleanup attempts:

Orphaned plists. A job definition whose `Program` no longer exists on disk means the payload was removed but the persistence was not. The plist names the file, giving you a path to hunt in snapshots, FSEvents and Spotlight.

Orphaned disable records. `<DATA>/private/var/db/com.apple.xpc.launchd/` holds `disabled.plist` and `disabled-<uid>.plist`, recording jobs explicitly enabled or disabled by label. Labels here that correspond to no plist on disk are the ghost of a job that once existed. Attackers routinely delete the plist and forget this file.

```
└──╼ $ plistutil -i /mnt/data/apfs5/private/var/db/com.apple.xpc.launchd/disabled.plist -o -
# older layout, worth checking on upgraded systems:
# private/var/db/launchd.db/com.apple.launchd/overrides.plist
```

35 BTM — the modern login-item store 13+

Ventura replaced the old login-items mechanism with Background Task Management. Every login item, launch agent, launch daemon and app-registered background task is recorded in a single database — and it retains entries for items that have since been removed.

Ventura+	`<DATA>/private/var/db/com.apple.backgroundtaskmanagement/BackgroundItems-v<N>.btm`
Pre-Ventura	`<DATA>/Users/<u>/Library/Application Support/com.apple.backgroundtaskmanagementagent/backgrounditems.btm`
Format	NSKeyedArchiver binary plist — see §33

```
└─╼ $ ls /mnt/data/apfs5/private/var/db/com.apple.backgroundtaskmanagement/
BackgroundItems-v9.btm  BackgroundItems-v8.btm

└─╼ $ python3 - <<'PY'
import nska_deserialize as nd, json, glob
for p in glob.glob('/mnt/data/apfs5/private/var/db/com.apple.backgroundtaskmanagement/*.btm'):
    with open(p,'rb') as f:
        obj = nd.deserialize_plist(f)
    print('===', p)
    print(json.dumps(obj, indent=2, default=str)[:4000])
PY
```

▲ DEVIL — BTM KEEPS THE DEAD

The BTM store records items with a developer team identifier, a bundle path, an item type, and a disposition (enabled, disabled, allowed, removed by user). **Entries persist after the item is deleted from disk.** On a machine that was cleaned up, BTM frequently still names the launch agent, its executable path and its signing team — everything you need to identify what was there.

Note also the versioned filenames. Older `BackgroundItems-v<N-1>.btm` files often survive alongside the current one, giving you a historical view of persistence that predates the attacker's cleanup by weeks.

36 Scheduled execution

Path	Notes
<DATA>/private/var/at/tabs/<user>	Per-user crontabs. Still fully functional and frequently unchecked
<DATA>/private/etc/crontab	System crontab
<DATA>/private/var/at/jobs/	at one-shot jobs
<DATA>/private/etc/periodic/{daily,weekly,monthly}/	Scripts run on a schedule by periodic
<DATA>/private/etc/periodic.conf, defaults/periodic.conf	Configuration — can point periodic at additional directories
<DATA>/private/etc/emond.d/	≤12 Removed in Ventura. **Its presence on 13+ is an anomaly worth investigating.**

```
└──╼ $ for f in /mnt/data/apfs5/private/var/at/tabs/*; do echo "== $f"; cat "$f"; done
└──╼ $ ls -la /mnt/data/apfs5/private/etc/periodic/*/
└──╼ $ cat /mnt/data/apfs5/private/etc/crontab
```

37 Profiles, MDM and system extensions

37.1 Configuration profiles

A profile can install certificates, force proxy settings, add login items, and grant TCC permissions without prompting. On an unmanaged personal machine, **any** profile deserves scrutiny.

```
└─╼ $ ls -la /mnt/data/apfs5/private/var/db/ConfigurationProfiles/Settings/
└─╼ $ ls -la /mnt/data/apfs5/private/var/db/ConfigurationProfiles/Store/
└─╼ $ ls -la "/mnt/data/apfs5/Library/Managed Preferences/"

# a profile installing a root CA is a full TLS interception capability
└─╼ $ grep -rl 'com.apple.security.root\|PayloadType.*certificate' \
    /mnt/data/apfs5/private/var/db/ConfigurationProfiles/ 2>/dev/null
```

37.2 System extensions and kext approval

```
# installed system extensions and their state
└─╼ $ plistutil -i /mnt/data/apfs5/Library/SystemExtensions/db.plist -o - | head -80
└─╼ $ ls /mnt/data/apfs5/Library/SystemExtensions/

# user-approved kernel extensions, with team identifiers — SQLite
└─╼ $ sqlite3 /mnt/data/apfs5/private/var/db/SystemPolicyConfiguration/KextPolicy \
  "SELECT team_id, bundle_id, allowed, flags FROM kext_policy;"
EQHXZ8M8AV|com.example.driver|1|8
```

▲ DEVIL — ExecPolicy IS A FIRST-EXECUTION LOG

Next to KextPolicy sits `ExecPolicy`, and it is one of the most under-used artifacts on macOS. Gatekeeper records executables the first time they run, with paths, signing information and timestamps — including for binaries that were later deleted.

```
└─╼ $ sqlite3 /mnt/data/apfs5/private/var/db/SystemPolicyConfiguration/ExecPolicy ".tables"
legacy_exec_history_v4  policy_scan_cache  scan_detail  ...

└─╼ $ sqlite3 /mnt/data/apfs5/private/var/db/SystemPolicyConfiguration/ExecPolicy "
SELECT datetime(timestamp,'unixepoch') AS first_seen, responsible_path, file_identifier
FROM policy_scan_cache ORDER BY timestamp DESC LIMIT 40;"
```

Read that as: **a list of programs that ran on this machine for the first time, with dates.** Cross-reference every path against what still exists on disk — the entries with no surviving file are your leads.

38 Plugin and injection surfaces

macOS loads third-party code into system processes through a long list of plugin directories. Each is a persistence mechanism that involves no launchd job at all, which is precisely why they get used.

Location (system and `~/Library` variants)	Loaded into / triggered by
/Library/Security/SecurityAgentPlugins/	**Authorisation plugins — run at the login window, as root.** Very high value
/Library/Audio/Plug-Ins/HAL/	coreaudiod — the classic microphone-capture persistence
/Library/QuickLook/	Thumbnail generation — triggered merely by viewing a folder
/Library/Spotlight/	Metadata importers — triggered by indexing
/Library/Input Methods/	Runs in every application that accepts text
/Library/ColorPickers/, /Library/PreferencePanes/	Loaded on demand by UI
/Library/Screen Savers/	Runs on idle
/Library/Internet Plug-Ins/	Legacy, still loaded by some applications
~/Library/Services/, ~/Library/Workflows/	Automator services and Folder Actions
App bundle Contents/XPCServices/, Contents/PlugIns/	Code inside a legitimate app's bundle — inherits its TCC grants

```
# sweep every plugin surface at once, newest first
└──╼ $ for d in "Security/SecurityAgentPlugins" "Audio/Plug-Ins/HAL" "QuickLook" "Spotlight" \
        "Input Methods" "ColorPickers" "PreferencePanes" "Screen Savers" "Internet Plug-Ins"; do
    for base in /mnt/data/apfs5/Library /mnt/data/apfs5/Users/*/Library; do
      [ -d "$base/$d" ] && find "$base/$d" -maxdepth 1 -mindepth 1 -printf '%TY-%Tm-%Td %p\n'
    done
  done | sort -r
```

▲ DEVIL — LOGIN AND LOGOUT HOOKS STILL WORK

Deprecated for over a decade, never removed. A single preference key runs an arbitrary script as root at every login or logout, with no plist in any LaunchAgents directory and no BTM entry.

```
└──╼ $ plistutil -i /mnt/data/apfs5/Library/Preferences/com.apple.loginwindow.plist -o - | \
    grep -A2 -i 'LoginHook\|LogoutHook'
```

In the same file, check `autoLoginUser` (automatic login without a password) and, in `~/Library/Preferences/ByHost/com.apple.loginwindow.<UUID>.plist`, `TALAppsToRelaunchAtLogin` — the "reopen windows when logging back in" list, which is another way to get an application launched at login with no persistence artifact.

39 Unix-side persistence

macOS is BSD underneath and every traditional Unix mechanism still functions. These are often skipped by macOS-focused triage tools.

Path	Check for
Users/<u>/.zshrc, .zshenv, .zprofile, .bash_profile	Appended commands, aliases shadowing real binaries, curl|sh
private/etc/zshenv, zprofile, profile	System-wide shell startup — applies to every user
private/etc/sudoers, sudoers.d/	NOPASSWD entries, unexpected users
Users/<u>/.ssh/authorized_keys	Attacker public keys — check from= and command restrictions too
private/etc/ssh/sshd_config, sshd_config.d/	PermitRootLogin, alternate ports, AuthorizedKeysFile redirection
private/etc/pam.d/	Added modules — a malicious PAM module captures credentials at every sudo
private/etc/hosts, resolv.conf	Redirection of update or security domains
usr/local/bin/, usr/local/sbin/	**Writable despite looking like a system path** (§12.3) — shadowing real commands
private/var/db/dslocal/nodes/Default/users/	Account records — hidden users, root enabled, added admins (§55)

```
# shell startup files modified after the OS was installed
└─╼ $ find /mnt/data/apfs5/Users/*/ -maxdepth 1 \
    \( -name '.zsh*' -o -name '.bash*' -o -name '.profile' \) \
    -printf '%TY-%Tm-%Td %T@ %p\n' | sort -k2 -rn

# anything in the shell files that fetches and executes
└─╼ $ grep -nE 'curl|wget|base64|osascript|python|nc |/dev/tcp' \
    /mnt/data/apfs5/Users/*/.z* /mnt/data/apfs5/Users/*/.bash* 2>/dev/null
```

40 The persistence sweep

One pass, in order. Tick them off; do not rely on memory.

Launch and login

- [ ] `/Library/LaunchDaemons`
- [ ] `/Library/LaunchAgents`
- [ ] `~/Library/LaunchAgents` (every user)
- [ ] `/System/Library/Launch*` (seal check)
- [ ] `/Library/StartupItems`
- [ ] BTM `BackgroundItems-v*.btm` + older versions
- [ ] `com.apple.xpc.launchd/disabled*.plist`
- [ ] Login/Logout hooks
- [ ] `TALAppsToRelaunchAtLogin`

Scheduled

- [ ] `/var/at/tabs/*`
- [ ] `/etc/crontab`, `/var/at/jobs`
- [ ] `/etc/periodic/*` + `periodic.conf`
- [ ] `/etc/emond.d` (anomaly on 13+)

Managed

- [ ] Configuration profiles
- [ ] Managed Preferences
- [ ] MDM enrollment state
- [ ] System extensions `db.plist`
- [ ] KextPolicy, **ExecPolicy**

Plugin surfaces

- [ ] SecurityAgentPlugins
- [ ] Audio HAL plug-ins
- [ ] QuickLook, Spotlight importers
- [ ] Input Methods, ColorPickers
- [ ] PreferencePanes, Screen Savers
- [ ] Services / Workflows / Folder Actions
- [ ] App bundle XPCServices & PlugIns

Unix

- [ ] Shell rc files, user and system
- [ ] `sudoers` + `sudoers.d`
- [ ] `authorized_keys`, `sshd_config`
- [ ] `pam.d`
- [ ] `hosts`, `resolv.conf`
- [ ] `/usr/local/{bin,sbin}`
- [ ] `dslocal` user records

Cross-checks

- [ ] Repeat the entire sweep **inside every snapshot**
- [ ] Diff snapshot results against the live volume
- [ ] Every plist's `Program` exists?
- [ ] Every binary code-signed? (§57)

> **■ CAUTION — THE SWEEP IS WORTHLESS WITHOUT THE SNAPSHOT PASS**
>
> A competent adversary removes their persistence before the machine is imaged. Running this checklist against the live volume alone will then return nothing, and you will report "no persistence identified" about a machine that was comprehensively owned. **Run it again inside every snapshot** (§42) and diff the results. The difference between the two is the cleanup, and the cleanup is the finding.

IX

THE EXOTIC CHAPTER

Everything that does not fit a checklist. Reconstructing filesystem states nobody saved, catching timestamps that lie, reading the attributes that survive deletion, and pulling evidence out of the volumes every other guide ignores. This part assumes Parts III and IV — it operates on structure, not on files.

41 Checkpoint time-travel
42 Snapshot archaeology
43 XID and inode anomaly detection
44 Orphans, tombstones and dangling extents
45 Free space and the space manager
46 Verifying the seal
47 Extended attribute archaeology
48 decmpfs and compressed payloads
49 Preboot and Recovery volume analysis
50 The VM volume
51 Firmlink confusion and path hiding
52 Slack, sibling links and oversized xattrs
53 Timestamp triangulation
54 Anti-forensics detection
55 Deleted accounts and identity remnants
56 Keychain extraction
57 Mach-O triage without Apple tooling
58 Entropy mapping the container
59 Honest limits

41 Checkpoint time-travel

The checkpoint descriptor area holds superseded container superblocks — complete roots of the filesystem as it existed at earlier transactions. Because APFS is copy-on-write, the trees those roots point at are frequently still intact on disk. Reconstructing one gives you the entire filesystem as it was, with names, paths and metadata, for a moment nobody chose to snapshot.

41.1 Parsing the container superblock

Field offsets, all little-endian, from the start of the block:

Offset	Size	Field	Meaning
0x00	8	o_cksum	Fletcher-64 over bytes 8..end
0x08	8	o_oid	Object id
0x10	8	o_xid	**Transaction id — how you order checkpoints**
0x18	4	o_type	Object type
0x20	4	nx_magic	'NXSB' = 4E 58 53 42
0x24	4	nx_block_size	Usually 4096
0x28	8	nx_block_count	Container size in blocks
0x60	8	nx_next_xid	Next transaction to be issued
0x68	4	nx_xp_desc_blocks	Size of the descriptor ring
0x70	8	nx_xp_desc_base	**Block address of the descriptor area**
0x78	8	nx_xp_data_base	Block address of the data area
0x98	8	nx_spaceman_oid	Space manager — §45
0xA0	8	nx_omap_oid	Container object map
0xB8	800	nx_fs_oid[100]	Volume superblock object ids

```
└─╼ $ cat > ~/cases/001/06-analysis/nxsb_scan.py <<'PY'
#!/usr/bin/env python3
"""Enumerate every valid container superblock in the checkpoint descriptor area."""
import struct, sys

IMG   = sys.argv[1]
START = int(sys.argv[2])        # container byte offset, from mmls
BS    = 4096

def fletcher64(data):
    """APFS variant: computed over the object excluding the 8-byte checksum."""
    lo = hi = 0
    for i in range(0, len(data), 4):
        lo = (lo + int.from_bytes(data[i:i+4], 'little')) % 0xFFFFFFFF
        hi = (hi + lo) % 0xFFFFFFFF
    c1 = (0xFFFFFFFF - ((lo + hi) % 0xFFFFFFFF))
    c2 = (0xFFFFFFFF - ((lo + c1) % 0xFFFFFFFF))
    return (c2 << 32) | c1

with open(IMG, 'rb') as f:
    f.seek(START)
    blk0 = f.read(BS)
    assert blk0[0x20:0x24] == b'NXSB', 'not an APFS container at that offset'
    desc_base   = struct.unpack_from('<Q', blk0, 0x70)[0]
    desc_blocks = struct.unpack_from('<I', blk0, 0x68)[0] & 0x7FFFFFFF
    print(f'descriptor area: block {desc_base}, {desc_blocks} blocks\n')

    found = []
    for i in range(desc_blocks):
        f.seek(START + (desc_base + i) * BS)
        b = f.read(BS)
        if b[0x20:0x24] != b'NXSB':
            continue
        xid    = struct.unpack_from('<Q', b, 0x10)[0]
        stored = struct.unpack_from('<Q', b, 0x00)[0]
        valid  = fletcher64(b[8:]) == stored
        found.append((xid, desc_base + i, valid))

for xid, blk, ok in sorted(found):
    print(f'xid {xid:12d}  block {blk:8d}  checksum {"OK" if ok else "BAD"}')
print(f'\n{len(found)} checkpoints recoverable')
PY

└─╼ $ python3 ~/cases/001/06-analysis/nxsb_scan.py /evidence/mbp-001.dd 524308480
descriptor area: block 193, 280 blocks

xid       214402  block      201  checksum OK
xid       214409  block      208  checksum OK
xid       214417  block      216  checksum OK

23 checkpoints recoverable
```

41.2 Mounting an earlier checkpoint

Writing a full B-tree walker for an arbitrary checkpoint is a project. There is a much cheaper route that gets you a mountable filesystem: **make an older checkpoint the newest one**, on a copy-on-write overlay, and let the existing tools do the work.

> **■ CAUTION — OVERLAY ONLY, NEVER THE IMAGE**
>
> This technique writes to the container. It must be performed against the device-mapper overlay from §4.3. Verify with `lsblk` that you are pointed at `/dev/mapper/evid-cow` before any `dd`, and re-verify the master hash afterwards.

```
# 1. build the COW overlay (see §4.3) and confirm the target
└─ $ lsblk /dev/mapper/evid-cow

# 2. neutralise every checkpoint newer than the one you want, by corrupting
#   their magic so the mounter skips them. Here: keep xid 214402 at block 201.
└─ # for blk in 208 216; do
     printf 'XXXX' | dd of=/dev/mapper/evid-cow bs=1 \
        seek=$((524308480 + blk*4096 + 0x20)) conv=notrunc
   done

# 3. copy the chosen older superblock over block 0 as well
└─ # dd if=/dev/mapper/evid-cow of=/tmp/old-nxsb.bin bs=4096 \
      skip=$(( (524308480/4096) + 201 )) count=1
└─ # dd if=/tmp/old-nxsb.bin of=/dev/mapper/evid-cow bs=4096 \
      seek=$(( 524308480/4096 )) count=1 conv=notrunc

# 4. mount normally — you are now looking at the filesystem at xid 214402
└─ # apfs-fuse -v 4 -p 'PW' -o ro /dev/mapper/evid-cow /mnt/past
└─ $ ls -la /mnt/past/Users/jdoe/Downloads/

# 5. tear down and prove the master is untouched
└─ # fusermount3 -u /mnt/past; dmsetup remove evid-cow
└─ $ sha256sum -c /evidence/mbp-001.dd.sha256
```

▲ DEVIL — WHAT THIS BUYS YOU, AND WHAT IT DOES NOT

Buys you: the complete filesystem — paths, names, timestamps, contents — at a moment in the recent past that no snapshot covers. Checkpoints are written constantly, so the window is typically minutes to hours before acquisition. If a file was deleted an hour before the machine was seized, this is frequently the only way to get it back with its metadata intact.

Does not buy you: anything whose blocks have been reallocated since. Older checkpoints degrade — the oldest entries in the ring will often fail to mount because their trees have been partly overwritten. Work newest-to-oldest and expect a cliff. Also note the descriptor ring is small: this is a window of hours, not weeks. For weeks, you need snapshots.

42 Snapshot archaeology

Snapshots are the single highest-yield artifact on APFS, and the one most likely to defeat an adversary's cleanup entirely.

42.1 Enumerate everything, on every volume

```
└─╼ $ for v in 1 2 3 4 5; do
    echo "=== volume $v"
    fsapfsinfo -o 524308480 -p 'PW' -f $v -s /evidence/mbp-001.dd 2>/dev/null | \
        grep -E 'Name|XID|Time'
  done
```

42.2 Mount them all and sweep each

```
└─╼ $ SNAPS=$(fsapfsinfo -o 524308480 -p 'PW' -f 5 -s /evidence/mbp-001.dd | \
       awk '/Name/{print $3}')
└─╼ $ i=0; for s in $SNAPS; do
    i=$((i+1)); mkdir -p /mnt/snap$i
    apfs-fuse -v 4 -p 'PW' -s "$s" -o ro /dev/loop0p2 /mnt/snap$i
    echo "$i -> $s"
  done

# run the §40 persistence sweep inside each, and hash the results
└─╼ $ for m in /mnt/snap*; do
    find $m/Library/LaunchDaemons $m/Library/LaunchAgents \
         $m/Users/*/Library/LaunchAgents -name '*.plist' 2>/dev/null | \
      sed "s|$m||" | sort > ~/cases/001/06-analysis/persist-$(basename $m).txt
  done

# the diff between the oldest snapshot and the live volume is the cleanup
└─╼ $ diff ~/cases/001/06-analysis/persist-snap1.txt \
     ~/cases/001/06-analysis/persist-live.txt
< /Users/jdoe/Library/LaunchAgents/com.apple.softwareupdated.plist
```

▸ **TIP — DIFF THE WHOLE TREE, NOT JUST PERSISTENCE**

A full recursive diff between a snapshot and the live volume is the fastest way to see everything that changed in that interval — files added, modified and deleted, all at once.

```
└─╼ $ diff -rq /mnt/snap1/Users /mnt/data/apfs5/Users 2>/dev/null | head -100
Only in /mnt/snap1/Users/jdoe/Downloads: installer.pkg
Files /mnt/snap1/Users/jdoe/.zsh_history and /mnt/data/.../\
.zsh_history differ
```

That second line is worth pausing on: the shell history was modified between the snapshot and acquisition. The snapshot holds the version before it was edited.

42.3 Detecting deleted snapshots

An adversary who understands APFS will delete the snapshots. That deletion is itself detectable.

- **Cadence gaps.** Time Machine local snapshots are created hourly and named with their timestamp. Sort the names; a missing hour in an otherwise unbroken hourly series, on a machine that was powered on through that hour (confirm via unified log or powerlog), means a snapshot was removed.
- **XID gaps.** Snapshot XIDs advance with the container. A large XID jump between consecutive surviving snapshots indicates transactions — and probably snapshots — in between.

- **Orphaned snapshot metadata.** Deletion removes the snapshot's records from the snapshot metadata tree, but the extents it pinned may not be immediately reclaimed. Blocks referenced by no live object and no surviving snapshot are the remains (§45).
- **The unified log records it.** Snapshot creation and deletion are logged. Grep the parsed unified log for `snapshot`, `APFSSnapshot`, `tmutil`, `localsnapshot`, `deletelocalsnapshots`.

```
└──╼ $ grep -iE 'snapshot|localsnapshot|tmutil' ~/cases/001/04-timeline/unifiedlog.csv | \
   grep -iE 'delet|destroy|remov' | head -30
```

43 XID and inode anomaly detection

The most reliable anti-timestomping technique available on APFS, and it requires nothing but metadata you already have.

43.1 The principle

Inode object identifiers are allocated sequentially from the volume superblock's `apfs_next_obj_id` counter. Every new file gets the next number. Therefore **object id order is creation order** — a fact no user-space tool can alter, because the counter is filesystem state, not file metadata.

```
Normal volume:  object id ▲ correlates with creation time ▲

obj_id  2100000 ──────────────────────────────── 2026-07-30  ✓
obj_id  2100001 ──────────────────────────────── 2026-07-30  ✓
obj_id  2100002 ──────────────────────────────── 2019-03-14  ✗  ← impossible
obj_id  2100003 ──────────────────────────────── 2026-07-30  ✓
```

A file with a high object id and an old creation date was created recently and backdated. There is no benign explanation on a single volume: the counter only moves forward.

43.2 Detecting it

```
└──╼ $ cat > ~/cases/001/06-analysis/xid_anomaly.py <<'PY'
#!/usr/bin/env python3
"""Flag files whose inode number is inconsistent with their creation time."""
import subprocess, re, sys, statistics

IMG, OFF, VOL = sys.argv[1], sys.argv[2], sys.argv[3]

# body file gives inode + crtime for every file in one pass
out = subprocess.run(['fls','-o',OFF,'-B',VOL,'-r','-m','/',IMG],
                     capture_output=True, text=True).stdout

rows = []
for line in out.splitlines():
    p = line.split('|')
    if len(p) < 11: continue
    try:
        inode  = int(p[2].split('-')[0])
        crtime = int(p[10])
    except ValueError:
        continue
    if crtime > 0:
        rows.append((inode, crtime, p[1]))

rows.sort()                       # by inode = by true creation order

# walk forward; creation time should be non-decreasing. Report inversions.
high_water = 0; anomalies = []
for inode, crtime, name in rows:
    if crtime < high_water - 86400:      # more than a day out of order
        anomalies.append((inode, crtime, high_water, name))
    high_water = max(high_water, crtime)

import datetime as dt
f = lambda t: dt.datetime.utcfromtimestamp(t).strftime('%Y-%m-%d %H:%M:%S')
print(f'{len(anomalies)} inode/timestamp inversions\n')
for inode, crtime, hw, name in anomalies[:60]:
    print(f'inode {inode:9d}  claims {f(crtime)}  '
          f'but neighbours are {f(hw)}\n    {name}')
PY

└──╼ $ python3 ~/cases/001/06-analysis/xid_anomaly.py /evidence/mbp-001.dd 1024040 1276
3 inode/timestamp inversions

inode   2100002  claims 2019-03-14 09:12:00  but neighbours are 2026-07-30 02:38:44
    /data/Users/jdoe/Library/LaunchAgents/com.apple.softwareupdated.plist
```

■ CAUTION — LEGITIMATE INVERSIONS EXIST

Before reporting a backdated file, rule out the innocent explanations:

- **Restores.** Migration Assistant, Time Machine restores and `rsync -a` recreate files with preserved original timestamps and fresh inode numbers. A restore produces thousands of inversions in one contiguous inode range — a distinctive, obviously benign pattern.
- **Archive extraction.** `tar -p`, `unzip` and installers preserve stored timestamps.
- **Clock changes.** If the system clock moved backwards, everything created afterwards inverts. Check the unified log for `settimeofday` first.

The finding is not "an inversion exists". It is "**an isolated inversion**, of a small number of files, in a persistence-relevant location, with no restore or clock event to explain it". That is what the output above shows: three anomalies, one of which is a launch agent.

43.3 Corroborating witnesses

Once you have a candidate, gather the independent evidence from §13.1 and §14:

Witness	What it should show for a stomped file
Inode number vs neighbours	Created recently, claims to be old
`change_time`	Recent — the moment the lie was written
`write_generation_counter`	Inconsistent with a file "untouched since 2019"
Containing directory's `mod_time`	Directory was modified when the file was really created
FSEvents	A creation event for that path, at the real time
Unified log / ExecPolicy	First-execution record at the real time
Snapshots	File absent from snapshots older than its claimed date

That last one is decisive and trivially checkable: a file claiming creation in 2019 that does not appear in a snapshot taken last week did not exist last week.

44 Orphans, tombstones and dangling extents

44.1 The private directory

APFS reserves two well-known object ids: **2** is the volume root directory, **3** is the private directory. Files that were unlinked while still open by a process are moved into the private directory rather than deleted — they have no name in any directory, but their inode and data remain fully intact until the last handle closes.

> **▲ DEVIL — DELETED-BUT-OPEN FILES SURVIVE A DEAD-BOX SEIZURE**
>
> If the machine was powered off abruptly — pulled plug, forced shutdown, battery removal — every file that a running process had unlinked but still held open is **sitting in the private directory, complete**. Malware that deletes itself from disk at launch and keeps running does exactly this. So does a logging process whose log was deleted underneath it.
>
> This is one of the few genuine advantages of an ungraceful shutdown, and almost nothing enumerates it automatically. Walk inodes whose `parent_id` is 3, or whose parent id resolves to no directory record at all.

```
└─╼ $ python3 - <<'PY'
import pyfsapfs
v = pyfsapfs.volume()
c = pyfsapfs.container(); c.open('/evidence/mbp-001.dd')   # offset handling per your build
# walk the file entries and report any whose parent is the private directory
def walk(entry, path=''):
    try: sub = entry.number_of_sub_file_entries
    except Exception: return
    for i in range(sub):
        ch = entry.get_sub_file_entry(i)
        yield ch, path + '/' + (ch.name or '?')
        yield from walk(ch, path + '/' + (ch.name or '?'))
PY

# practical alternative: TSK lists allocated inodes with no directory entry
└─╼ $ fls -o $OFF -B $DAT -r -p $IMG | awk '{print $2}' | sort -u > /tmp/named.txt
└─╼ $ istat -o $OFF -B $DAT $IMG 3       # the private directory itself
```

44.2 Dangling extents

A `FILE_EXTENT` record maps a range of blocks to a file. When records are removed inconsistently — through corruption, interrupted deletion, or deliberate tampering — you get extents referencing blocks that no live inode claims. Those blocks hold real file content with no name attached.

```
# apfsck reports structural inconsistencies including orphaned records
└─╼ # apfsck -c /dev/mapper/evid-cow
# run against the OVERLAY: apfsck can be told to repair, and you never want that
# touching the evidence
```

45 Free space and the space manager

On a copy-on-write filesystem, unreferenced blocks are where every superseded version of every file lives. Extracting them properly means reading the space manager's allocation bitmaps rather than trusting a tool's idea of "unallocated".

> ■ **CAUTION — THIS DOES NOT WORK ON THE ENCRYPTED VOLUME**
>
> Free blocks belonging to a FileVault-encrypted volume contain ciphertext encrypted with per-file keys that are gone along with their inodes. There is nothing to recover: the plaintext is unrecoverable even with the volume password, because the per-file key was destroyed with the file's crypto state record.
>
> Free-space work is therefore productive on the **unencrypted** volumes only — System, Preboot, Recovery, VM — and on any unencrypted external media images you hold. Know this before you spend a day on it. It is the single most common wasted effort on a FileVault case.

45.1 Reading the allocation bitmap

```
# the space manager object id comes from the container superblock at 0x98
└──╼ $ python3 - <<'PY'
import struct
IMG='/evidence/mbp-001.dd'; START=524308480; BS=4096
f=open(IMG,'rb'); f.seek(START); b=f.read(BS)
print('spaceman oid :', struct.unpack_from('<Q', b, 0x98)[0])
print('omap oid     :', struct.unpack_from('<Q', b, 0xA0)[0])
print('block count  :', struct.unpack_from('<Q', b, 0x28)[0])
PY
```

The space manager points at chunk-info blocks, each of which points at a bitmap block covering a run of container blocks — one bit per block, set meaning allocated. Walking that structure gives you an authoritative free list. For unencrypted volumes, extract those blocks and treat the result as a carving target:

```
# once you have a list of free block addresses in freeblocks.txt
└──╼ $ while read blk; do
     dd if=/evidence/mbp-001.dd bs=4096 skip=$(( (524308480/4096) + blk )) \
        count=1 status=none
   done < freeblocks.txt > ~/cases/001/05-carve/freespace.raw

└──╼ $ bulk_extractor -o ~/cases/001/05-carve/freespace-be ~/cases/001/05-carve/freespace.raw
└──╼ $ photorec /d ~/cases/001/05-carve/freespace-pr ~/cases/001/05-carve/freespace.raw
```

> ▸ **TIP — TRIAGE BY ENTROPY BEFORE CARVING**
>
> Most free space is zeroes or ciphertext. Measure before you carve: split the extracted free space into chunks, compute entropy per chunk, and only carve the regions that look like structured data. §58 has the method. It routinely turns an overnight carve into a ten-minute one.

46 Verifying the seal

The sealed System volume carries a SHA-256 for every file it contains, and a Merkle root over the whole set. This gives you two capabilities: proving the OS was not modified, and — more usefully — a complete, version-exact known-good hash set.

46.1 Read the stored seal and the broken flag

```
└─╼ $ fsapfsinfo -o 524308480 -f 1 /evidence/mbp-001.dd | grep -i -A6 'integrity'
        Integrity metadata:
                Version           : 1
                Hash type         : SHA-256 (1)
                Root hash         : 4f9c2a...e71b
                Flags             : 0x00000000
# flags bit 0 = APFS_SEAL_BROKEN. Non-zero here is a finding on its own.
```

46.2 Verify files against their FILE_INFO hashes

```
# hash every file on the mounted System volume
└─╼ $ find /mnt/sys/apfs1 -type f -print0 | \
    xargs -0 -P4 sha256sum > ~/cases/001/06-analysis/system-actual.sha256

# compare against the hashes APFS itself stores per file.
# fsapfsinfo exposes them per entry; mac_apt's plugin does this in bulk.
└─╼ $ python3 ~/tools/mac_apt/mac_apt.py -o ~/cases/001/06-analysis/macapt \
    MOUNTED /mnt/sys/apfs1 UNIFIEDLOGS BASICINFO
```

▲ DEVIL — A MODIFIED SYSTEM BINARY IS NOT SUBTLE, AND THAT IS THE POINT

Modifying the sealed volume requires disabling SIP, booting to recovery, and reducing security — all of which leave records (unified log, Preboot LocalPolicy, NVRAM). An attacker who does it anyway is either very confident or very careless. Either way the mismatch between a file's actual SHA-256 and its stored `FILE_INFO` hash names the modified file precisely.

Remember your own footprint. You disabled SIP during acquisition (§1.2). That does not break the seal by itself — it changes the boot policy, not the volume — but confirm the timing of any Preboot changes against your acquisition record before attributing them to anyone else.

46.3 Using the seal as a hash set

The more valuable everyday use. Every file on the System volume has an authoritative hash placed there by Apple for this exact build. Extract them all and use the set to eliminate known-good files across your entire examination — more precise than NSRL, and already on your image.

47 Extended attribute archaeology

macOS hangs a remarkable amount of history off files as extended attributes. Most examinations look at exactly one of them.

Attribute	What it records
com.apple.quarantine	Downloading agent, timestamp, event UUID (§27)
com.apple.metadata:kMDItemWhereFroms	Source URL and referrer, as a binary plist
com.apple.macl	**Which applications the user granted access to this file** — see below
com.apple.lastuseddate#PS	Last-opened time, maintained by Launch Services independently of atime
com.apple.FinderInfo	Finder flags, including the hidden bit and legacy type/creator codes
com.apple.ResourceFork	Resource fork content — including compressed data (§48)
com.apple.provenance	Installation provenance on newer systems
com.apple.TextEncoding	Encoding hint — occasionally reveals a document's origin locale
com.apple.diskimages.*	On DMGs: checksum state and last verification

```
# dump every xattr on every file in a tree, base64 for binary values
└─╸ $ getfattr -R -d -m '.*' -e base64 /mnt/data/apfs5/Users/jdoe/ \
    > ~/cases/001/06-analysis/xattrs-jdoe.txt 2>/dev/null

# which attributes exist at all, ranked — a good orientation step
└─╸ $ grep -oP '^[a-z0-9.#]+(?==)' ~/cases/001/06-analysis/xattrs-jdoe.txt | \
    sort | uniq -c | sort -rn
  48211 com.apple.lastuseddate#PS
   9033 com.apple.quarantine
   4120 com.apple.macl
    881 com.apple.metadata:kMDItemWhereFroms
```

▲ DEVIL — com.apple.macl NAMES THE APPLICATION THAT OPENED THE FILE

When a user opens a file through a sandboxed app's file dialog, macOS writes an entry into `com.apple.macl` recording the UUID of the application granted access. It is not a timestamp and not a log — it is a durable record that **this specific application was given access to this specific file**, and it **survives copying the file to another location and another volume**.

Practical uses: proving a document was opened by a particular application even though the app's own recent-items list was cleared; establishing that a file copied to a USB stick had previously been opened on this machine; and linking a file to an application that has since been deleted. Almost nothing in the standard toolchain surfaces it — you have to ask for it by name.

```
└─╸ $ getfattr -n com.apple.macl -e hex --only-values \
    /mnt/data/apfs5/Users/jdoe/Documents/plans.pdf
```

▸ **TIP — lastuseddate#PS IS A BETTER "OPENED" TIMESTAMP THAN atime**

Access times on macOS are unreliable — updated inconsistently, and clobbered by backup and indexing processes touching every file. `com.apple.lastuseddate#PS` is written by Launch Services when a document is actually opened by a user, and it is a much stronger indicator of human interaction. It is a little-endian `timespec`: 8 bytes of Unix seconds, 8 bytes of nanoseconds.

```
└──╼ $ getfattr -n 'com.apple.lastuseddate#PS' -e hex --only-values file.pdf | \
  sed 's/^0x//' | head -c16 | \
  python3 -c "import sys,struct,datetime;
d=bytes.fromhex(sys.stdin.read());
print(datetime.datetime.utcfromtimestamp(struct.unpack('<q',d)[0]))"
```

48 decmpfs and compressed payloads

macOS transparently compresses files, storing the compressed data either inline in the `com.apple.decmpfs` extended attribute or in the resource fork. The file reports its uncompressed size and has **no data extents at all**.

Type	Algorithm	Storage
3 / 4	zlib	3 = inline in xattr, 4 = resource fork
7 / 8	LZVN	7 = inline, 8 = resource fork
11 / 12	LZFSE	11 = inline, 12 = resource fork

▲ DEVIL — A FILE WITH NO EXTENTS THAT IS NOT EMPTY

Two consequences that bite:

Tools disagree. `apfs-fuse` decompresses transparently and shows you the real content. TSK and some libfs-apfs paths may report the file's logical size but return no data, making it look like an empty or sparse file. If `icat` gives you nothing for a file that `apfs-fuse` shows as 40 KB of text, it is compressed — not deleted, not corrupt.

It is a hiding place. Content stored in a resource fork or decmpfs attribute is invisible to anything that only walks data extents — including most carvers and any script that reads block allocations. Sweep for the attribute explicitly rather than assuming your tools handled it.

```
└──╼ $ getfattr -R -n com.apple.decmpfs --only-values /mnt/data/apfs5/Users/ 2>/dev/null | head
# files carrying a resource fork are worth enumerating too
└──╼ $ getfattr -R -m 'com.apple.ResourceFork' /mnt/data/apfs5/Users/ 2>/dev/null | \
    grep '^# file' | head -40
```

49 Preboot and Recovery volume analysis

Two unencrypted volumes that most examinations skip entirely. They are small, they are rarely modified, and that combination makes any modification meaningful.

49.1 Preboot

```
└──╼ $ fsapfsmount -o 524308480 -f 2 /evidence/mbp-001.dd /mnt/preboot
└──╼ $ find /mnt/preboot/apfs2 -maxdepth 3 | head -40
/mnt/preboot/apfs2/<DataVolumeUUID>
/mnt/preboot/apfs2/<DataVolumeUUID>/System/Library/CoreServices
/mnt/preboot/apfs2/<DataVolumeUUID>/var/db
/mnt/preboot/apfs2/<DataVolumeUUID>/LocalPolicy
/mnt/preboot/apfs2/Cryptexes
```

▲ DEVIL — LocalPolicy RECORDS THE SECURITY DOWNGRADE

On Apple Silicon, `LocalPolicy/*.img4` encodes the machine's boot security posture: Full, Reduced or Permissive security; whether SIP is disabled; whether third-party kexts are allowed; whether the boot policy permits an unsealed system volume.

A machine in **Reduced or Permissive security** was deliberately downgraded by someone with physical access and the admin password. If your case involves a modified system volume, disabled SIP, or unsigned kernel extensions, this is the corroborating artifact — and it is on an unencrypted volume you can read without any password.

Your own acquisition modified this. You ran `csrutil disable` (§1.2), which rewrites LocalPolicy. Compare timestamps against your acquisition record before attributing the downgrade to the subject. If the downgrade predates your acquisition, it was not you.

```
└──╼ $ find /mnt/preboot/apfs2 -name '*.img4' -printf '%TY-%Tm-%Td %T@ %p\n' | sort -k2 -n
```

Also on Preboot: per-volume boot support files, and `var/db/` containing volume ownership records — which identify the accounts permitted to unlock and boot the machine. An unexpected entry there is an account with FileVault unlock rights.

49.2 Recovery

recoveryOS changes only when macOS is updated. Enumerate it, note the timestamps, and treat any file whose modification time does not correspond to an OS update in `install.log` as requiring an explanation. Persistence installed here survives an erase-and-reinstall of the main system, which is exactly why sophisticated actors like it.

```
└──╼ $ fsapfsmount -o 524308480 -f 3 /evidence/mbp-001.dd /mnt/recovery
└──╼ $ find /mnt/recovery/apfs3 -type f -printf '%TY-%Tm-%Td %p\n' | sort | tail -30
# cross-check the dates against OS update events
└──╼ $ grep -i 'macOS.*Install\|softwareupdate' /mnt/data/apfs5/private/var/log/install.log | tail
```

50 The VM volume

Swap and hibernation. Historically a goldmine of process memory; on modern hardware, much less so — but worth ten minutes.

```
└──╼ $ fsapfsmount -o 524308480 -f 4 /evidence/mbp-001.dd /mnt/vm
└──╼ $ ls -la /mnt/vm/apfs4/
-rw------- 1 root root 1073741824 Jul 30 02:11 swapfile0
-rw------- 1 root root 1073741824 Jul 30 02:38 swapfile1
-rw------- 1 root root 8589934592 Jul 29 23:04 sleepimage
```

> ■ **CAUTION — SET YOUR EXPECTATIONS CORRECTLY**
>
> macOS encrypts swap by default, and on Apple Silicon the sleepimage is encrypted with a key held in the Secure Enclave that is discarded at wake. **You will usually find high-entropy noise and nothing else.** Do not spend a day on it, and do not report "swap was encrypted" as though it were an adverse finding — it is the default configuration.

What is still worth extracting:

- **Timestamps and sizes.** Swapfile count and modification times indicate memory pressure and roughly when the machine was under load. The sleepimage's timestamp is the last time the machine hibernated — a reliable "machine was here, doing this, at this time" data point.
- **Entropy check, cheaply.** If entropy is not uniformly high, something is unencrypted and worth looking at.

```
└──╼ $ ent /mnt/vm/apfs4/swapfile0 | head -3
Entropy = 7.999812 bits per byte.
# ~8.0 means encrypted or compressed — nothing to extract

# if entropy is lower, sweep it properly
└──╼ $ bulk_extractor -o ~/cases/001/05-carve/vm /mnt/vm/apfs4/swapfile0
└──╼ $ strings -n 12 /mnt/vm/apfs4/swapfile0 | grep -iE 'https?://|@|BEGIN (RSA|OPENSSH)' | head
```

51 Firmlink confusion and path hiding

§12.3 covered firmlinks as a translation problem. They are also a hiding technique, because the unified view macOS presents is a subset of what is physically on the two volumes.

51.1 Shadowed directories

When `/Library` is firmlinked to the Data volume's `Library`, the System volume's own `Library` at that path is shadowed — physically present, never shown. The same applies to `/Users`, `/opt`, `/usr/local` and every other firmlink target.

▲ DEVIL — CONTENT VISIBLE ONLY WHEN THE VOLUMES ARE SEPARATED

Files placed in a shadowed location are invisible to every live-system triage tool, to `find /`, to Spotlight, and to any script that walks the running filesystem. They are plainly visible to you, because you mounted the two volumes separately.

This is an advantage of dead-box analysis that live response cannot match. Enumerate both volumes independently and compare against the firmlink table — anything in a shadowed path is, by definition, something the running system was never showing anyone.

```
# what exists on the System volume at firmlinked paths — normally near-empty
└─╼ $ for p in Library Users Applications opt cores; do
      echo "== /$p on SYSTEM volume"; ls -la /mnt/sys/apfs1/$p 2>/dev/null | head
    done
```

Note that the System volume is sealed, so writing there breaks the seal (§46) — which makes this a loud technique, not a quiet one. The quiet version is the Data volume's own shadowed paths, which are not sealed at all.

51.2 Ordinary hiding, which is more common

```
# BSD hidden flag — invisible in Finder, visible to you
└─╼ $ find /mnt/data/apfs5/Users -flags +hidden 2>/dev/null | head
# (Linux find lacks -flags; read bsd_flags via fsapfsinfo or check UF_HIDDEN=0x8000)

# names designed to be overlooked
└─╼ $ find /mnt/data/apfs5/Users -name '.*' -type f | grep -vE '\.(DS_Store|localized|CFUserText)' | head
-40
└─╼ $ find /mnt/data/apfs5 -name '* ' -o -name ' *' 2>/dev/null | head   # trailing/leading spaces

# non-ASCII and homoglyph filenames
└─╼ $ find /mnt/data/apfs5/Users -print0 | perl -0ne 'print if /[^\x00-\x7F]/' | head -30

# deeply nested or oddly-timestamped directories under /private/var/folders
└─╼ $ find /mnt/data/apfs5/private/var/folders -type f -newermt '2026-07-25' \
    -not -path '*Cache*' 2>/dev/null | head -40
```

52 Slack, sibling links and oversized xattrs

52.1 Hard links as an anti-deletion technique

APFS records hard links through sibling-link and sibling-map records: one inode, several names. Deleting one name decrements the link count; the data survives while any name remains.

```
# files with more than one link — rare on a normal Mac outside a few system areas
└─• $ find /mnt/data/apfs5 -type f -links +1 -printf '%n %i %p\n' 2>/dev/null | \
    sort -rn | head -40
```

> **▲ DEVIL — "DELETED" DATA WITH A SECOND NAME**
>
> A second hard link in an obscure location means that deleting the obvious copy destroys nothing. Conversely, if you find an inode with a link count higher than the number of names you can locate, there is a name you have not found — possibly in a shadowed path (§51), possibly in a snapshot. Both are worth chasing.

52.2 Oversized extended attributes

Small xattrs live inline in the catalog record. Once an attribute exceeds roughly 3.8 KB, APFS promotes it to its own data stream — meaning an extended attribute can hold arbitrarily large content that does not appear in the file's size, does not appear in directory listings, and is skipped by almost every carver and copying tool.

```
# rank every xattr on a tree by value size — outliers first
└─• $ getfattr -R -d -m '.*' -e base64 /mnt/data/apfs5/Users/ 2>/dev/null | \
  awk -F= '/=0s/ {print length($2), $1}' | sort -rn | head -20
41208 com.apple.ResourceFork
39114 com.apple.decmpfs
 8822 com.apple.macl
```

Anything unexpectedly large, or any attribute with a name you do not recognise, deserves extraction and a `file` check. This is a genuinely obscure hiding place and it survives copying the file with tools that preserve xattrs.

52.3 Slack space, honestly

APFS allocates in block units, so a file's final block has unused tail bytes. On a spinning disk that tail would hold residue from a previous file. On an SSD with TRIM — which is every machine you will examine — freed blocks are discarded by the controller and typically read back as zeroes, and APFS itself zero-fills in many paths. **Treat slack recovery on modern Mac hardware as a long shot, not a technique.** Check it if it is cheap; do not build a theory on it.

53 Timestamp triangulation

Any single timestamp can be forged. The defence is that macOS records the same event in many independent places, most of which an attacker does not know about and cannot easily reach.

53.1 The witness matrix

Witness	Granularity	Can it be forged?
Inode `create/mod/access`	nanosecond	**Yes** — trivially, via API
Inode `change_time`	nanosecond	Hard — no supported API
Inode number ordering	ordinal	**No** — filesystem-allocated (§43)
Object `o_xid`	ordinal	**No**
`write_generation_counter`	count	**No**
Parent directory `mod_time`	nanosecond	Yes, but usually forgotten
FSEvents	~second, ordered	Only by deleting the store wholesale
Unified log	microsecond	Very difficult — append-only, binary, signed structure
Snapshot membership	snapshot interval	**No** — the file is in it or it is not
ExecPolicy first-run	second	Requires root and DB surgery
Quarantine xattr + DB	second	Both must be forged consistently
Spotlight `kMDItem*` dates	second	Indirect; index rebuild changes them
SQLite `rowid` ordering	ordinal	**No** — monotonic per table
`install.log`	second	Plain text — editable, but rotation copies are not

▸ **TIP — SQLITE ROWIDS ARE A MONOTONIC CLOCK**

The same logic as inode numbers, applied to databases. In `History.db`, `chat.db`, `knowledgeC.db` and the rest, `rowid` increases with insertion order. A row bearing an old timestamp but a high `rowid` was inserted recently and backdated. One query finds them:

```
└──╼ $ sqlite3 History.db "
SELECT id, datetime(visit_time+978307200,'unixepoch') AS t
FROM history_visits ORDER BY id;" | \
  awk -F'|' 'NR>1 && $2 < prev {print "INVERSION at rowid", $1, $2, "after", prev} {prev=$2}'
```

53.2 The nanosecond tell

▲ **DEVIL — ZEROED NANOSECONDS**

Timestamps written by the kernel during normal filesystem activity carry a full nanosecond component — effectively random. Timestamps set deliberately through most tooling (`touch -t`, `SetFile -d`, and the majority of timestomping utilities) specify whole seconds, leaving the nanosecond field **exactly zero**.

Across a normal volume, files with all-zero nanoseconds on all four timestamps are a small minority. A cluster of them, in one directory, on files relevant to your case, is a strong signal.

```
└──╼ $ awk -F'|' '$9%1==0 && $10%1==0 {print $2}' body.txt | head
# better: read nanoseconds directly, where the body file truncates them
└──╼ $ stat --printf='%y|%z|%w|%n\n' /mnt/data/apfs5/Users/jdoe/Library/LaunchAgents/*.plist
2019-03-14 09:12:00.000000000 +0000|...      ← all zeroes: set, not written
```

53.3 Working a contradiction

The method, in order:

1. Fix the claim. "This file was created 2019-03-14."
2. Ask each witness independently. Inode ordering, change_time, FSEvents, snapshots, ExecPolicy, unified log, parent directory.
3. Count agreements. One dissenting witness is usually a parser or timezone error. Three or more dissenting, all pointing at the same alternative time, is a finding.
4. Explain the mechanism. "The inode number places creation between two files created at 02:38 on 2026-07-30; the change time is 02:39 the same day; the file is absent from the 02:15 snapshot and present in the 03:15 one." That is a defensible statement. "The timestamp looked wrong" is not.

54 Anti-forensics detection

You are looking for the shape of an absence. Each technique below leaves a characteristic hole.

Technique	What it leaves	How to detect
Shell history cleared	Truncated or missing `.zsh_history`; `HISTFILE` unset in rc files	Compare against snapshots; check file size vs account age; look for `unset HISTFILE` in `.zshrc`
Unified logging disabled	`log config` subsystem changes; abrupt reduction in log volume	Plot records per hour; a step change is not natural
Log files deleted	Gap in `.tracev3` sequence; `diagnostics/` newer than the volume	§24; compare against snapshots
FSEvents wiped	`fseventsd-uuid` differs from snapshot copies; store starts abruptly	Compare UUID and earliest event id across snapshots
Snapshots deleted	Cadence and XID gaps	§42.3
Timestamps stomped	Inode inversions; zero nanoseconds; late `change_time`	§43, §53
xattrs stripped	Downloaded files with no quarantine attribute	§27; compare siblings in the same folder
TCC reset	`tccutil reset` logged; grant history truncated	Unified log; compare TCC.db against snapshot copies
Files securely erased	Nothing in free space; but ExecPolicy, FSEvents, Spotlight and BTM still name them	Hunt the references, not the content
Clock manipulation	`settimeofday` in unified log; timeline inversions across many artifacts at once	Grep the log; look for a whole-system time discontinuity
Trash emptied	`.Trash` empty but FSEvents and `.DS_Store` retain names	§25, §31

▲ DEVIL — THE CLEANUP IS LOUDER THAN THE INTRUSION

Deleting evidence is itself an event, and it happens after the activity it conceals — which means it is the most recent thing on the machine and sits at the top of every timeline. An adversary who removes a launch agent, clears a history file and deletes a snapshot generates three FSEvents records, several unified log entries, a directory mtime update and a BTM disposition change, all within the same few minutes.

When you cannot find the intrusion, stop looking for it and look for the cleanup instead. Then work backwards from its timestamp.

54.1 Log volume as a detector

```
# records per hour from the parsed unified log — plot the shape of the day
└─╼ $ awk -F',' '{print substr($1,1,13)}' ~/cases/001/04-timeline/unifiedlog.csv | \
    uniq -c | sort -k2 | tail -48
  18422 2026-07-30 00
  17903 2026-07-30 01
    212 2026-07-30 02      ← two orders of magnitude drop
  18110 2026-07-30 04
```

A machine that was awake and busy does not emit two hundred log records in an hour. Correlate the gap against powerlog and knowledgeC: if the machine was asleep, the drop is expected; if it was in active use, something suppressed or removed logging for that hour.

55 Deleted accounts and identity remnants

Local accounts live in Directory Services' local node as one binary plist per user.

```
└──• $ ls /mnt/data/apfs5/private/var/db/dslocal/nodes/Default/users/
_amavisd.plist  daemon.plist  jdoe.plist  nobody.plist  root.plist

└──• $ plistutil -i /mnt/data/apfs5/private/var/db/dslocal/nodes/Default/users/jdoe.plist -o - | \
    grep -A2 -E 'uid|home|shell|realname|generateduid|authentication_authority'
```

55.1 Finding the accounts that are gone

```
# 1. UIDs that own files but have no account record
└──• $ find /mnt/data/apfs5/Users -maxdepth 3 -printf '%U\n' 2>/dev/null | sort -u
0
501
502          ← no plist for uid 502

└──• $ find /mnt/data/apfs5 -uid 502 2>/dev/null | head -40

# 2. the deleted-users holding area
└──• $ ls -la '/mnt/data/apfs5/Users/Deleted Users/' 2>/dev/null

# 3. admin group membership may still name them
└──• $ plistutil -i /mnt/data/apfs5/private/var/db/dslocal/nodes/Default/groups/admin.plist -o -

# 4. and the account record itself will be in older snapshots
└──• $ ls /mnt/snap1/private/var/db/dslocal/nodes/Default/users/
```

▲ DEVIL — THE PASSWORD HASH IS IN THE ACCOUNT PLIST

`ShadowHashData` inside the user plist holds the SALTED-SHA512-PBKDF2 verifier — salt, iteration count and hash. It is a legitimate target for offline cracking (`hashcat` mode **7100**) where you have authority to do so, and it is also an identity artifact in its own right: the same hash appearing on two machines links the accounts.

Practically, for your case the more useful angle is that a **deleted account's plist survives in snapshots**, complete with its GeneratedUID. That UID appears throughout the system — in TCC records, in keychain ACLs, in file ownership — letting you attribute activity to an account that no longer exists.

56 Keychain extraction

You have the user password, which makes the login keychain directly readable.

Keychain	Unlocked by
<DATA>/Library/Keychains/System.keychain	<DATA>/private/var/db/SystemKey — a file on disk, no password needed
<DATA>/Users/<u>/Library/Keychains/login.keychain-db	The user's login password
<DATA>/Users/<u>/Library/Keychains/<UUID>/keychain-2.db	Data-protection keychain — needs SEP-held keys, generally not recoverable dead-box

```
└─╼ $ cd ~/tools/chainbreaker

# the system keychain — unlocked with the on-disk SystemKey, no credential required
└─╼ $ python3 chainbreaker.py --dump-all \
    --key-file /mnt/data/apfs5/private/var/db/SystemKey \
    /mnt/data/apfs5/Library/Keychains/System.keychain \
    > ~/cases/001/06-analysis/system-keychain.txt

# the login keychain, with the password you already hold
└─╼ $ python3 chainbreaker.py --dump-all --password 'TheUserPassword' \
    /mnt/data/apfs5/Users/jdoe/Library/Keychains/login.keychain-db \
    > ~/cases/001/06-analysis/login-keychain.txt

└─╼ $ grep -iE 'internet password|generic password|Account:|Server:' \
    ~/cases/001/06-analysis/login-keychain.txt | head -40
```

■ CAUTION — SCOPE AND HANDLING

A keychain dump is a complete set of the subject's credentials: banking, email, VPN, Wi-Fi, certificates and private keys. Confirm this is within the authorised scope of your examination before running it, store the output with the same care as the evidence itself, and never place raw credentials in a report — reference them by service and account name only.

▸ TIP — THE SYSTEM KEYCHAIN NEEDS NO PASSWORD AT ALL

`System.keychain` is unlocked by `/var/db/SystemKey`, which is simply a file on the volume. Anyone with the decrypted disk — including you, and including an attacker with root — can read every Wi-Fi password and system credential without knowing anyone's password. Worth remembering both as a capability and as a risk you may need to explain in a report.

57 Mach-O triage without Apple tooling

You have no `codesign`, no `otool`, no `spctl`. Everything below runs on Linux.

57.1 Structure and entitlements with LIEF

```
└─╼ $ pip install lief
└─╼ $ python3 - <<'PY'
import lief, sys
p = '/mnt/data/apfs5/Users/jdoe/Library/App/updater'
fat = lief.MachO.parse(p)
for b in fat:
    print('---', b.header.cpu_type)
    print('  libraries:')
    for lib in b.libraries:
        print('   ', lib.name)
    if b.has_code_signature:
        print('  code signature present, size',
              b.code_signature.data_size)
    else:
        print('  *** NO CODE SIGNATURE ***')
    for cmd in b.commands:
        if cmd.command == lief.MachO.LOAD_COMMAND_TYPES.RPATH:
            print('  rpath:', cmd.path)
PY
```

57.2 Actually verifying a signature

The Rust `apple-codesign` crate ships `rcodesign`, which verifies Apple code signatures and prints entitlements — on Linux.

```
└─╼ $ cargo install apple-codesign
└─╼ $ rcodesign print-signature-info /mnt/data/apfs5/Applications/Suspect.app
└─╼ $ rcodesign verify /mnt/data/apfs5/Applications/Suspect.app
```

Finding	Interpretation
No signature at all	Cannot have passed Gatekeeper normally. Ran via an explicit override, a terminal, or with quarantine stripped
Ad-hoc signature	Self-signed with no identity — common for compiled-on-the-box tooling
Valid Developer ID	Note the Team ID and cross-reference against KextPolicy, TCC and ExecPolicy entries
`com.apple.security.cs.disable-library-validation`	Permits loading unsigned libraries — an injection enabler
`com.apple.security.cs.allow-dyld-environment-variables`	Permits `DYLD_INSERT_LIBRARIES` against this binary
`com.apple.security.get-task-allow`	Debuggable — permits another process to attach and read memory

```
# capability triage — maps behaviour to ATT&CK without running anything
└─╼ $ pip install flare-capa && capa /path/to/binary

# quick structural sweep of every Mach-O in user space
└─╼ $ find /mnt/data/apfs5/Users -type f -perm -u+x -exec file {} + 2>/dev/null | \
    grep -i 'Mach-O' | cut -d: -f1 > ~/cases/001/06-analysis/user-machos.txt
└─╼ $ wc -l ~/cases/001/06-analysis/user-machos.txt
```

58 Entropy mapping the container

A cheap way to find the interesting regions of a very large image: measure randomness and look at what does not fit its surroundings.

```
└─╼ $ cat > ~/cases/001/06-analysis/entropy_map.py <<'PY'
#!/usr/bin/env python3
import sys, math, collections
PATH  = sys.argv[1]
CHUNK = int(sys.argv[2]) if len(sys.argv) > 2 else 1 << 20   # 1 MiB

def entropy(b):
    if not b: return 0.0
    c = collections.Counter(b); n = len(b)
    return -sum((v/n) * math.log2(v/n) for v in c.values())

with open(PATH, 'rb') as f:
    off = 0
    while True:
        d = f.read(CHUNK)
        if not d: break
        e = entropy(d)
        bar = '#' * int(e * 6)
        print(f'{off:014d}  {e:5.3f}  {bar}')
        off += len(d)
PY

└─╼ $ python3 ~/cases/001/06-analysis/entropy_map.py /evidence/mbp-001.dd > entropy.txt

# regions that are neither empty nor encrypted — structured data worth carving
└─╼ $ awk '$2 > 3.0 && $2 < 7.0 {print}' entropy.txt | head -40
```

Entropy	Usually means
0.0 - 1.0	Zeroes or a single repeated byte — unused or wiped
3.0 - 6.0	Text, XML, source, SQLite, plists — **carve here**
6.0 - 7.5	Executables, images, mixed structured content
7.9 - 8.0	Encrypted or compressed. Uniform across the encrypted volume; interesting only when it appears somewhere it should not

▸ TIP — LOOK FOR THE INVERSION

The productive question is not "where is the high entropy" — on a FileVault machine that is everywhere. It is: **where is the low entropy inside a region that should be uniformly high?** A run of plaintext inside the encrypted volume's block range means something was written outside the encryption path. Equally, a high-entropy blob sitting inside an otherwise ordinary user directory is a packed binary, an archive, or an encrypted container worth identifying.

59 Honest limits

A short chapter, and the most important one for your report. These are the questions this image cannot answer, and saying so plainly is what makes the rest of your findings credible.

Memory	Gone. Running processes, injected code, decrypted keys, network connections and anything that never touched disk are unrecoverable. Volatility has no use here.
NVRAM and firmware	Not in the image. Boot arguments, EFI variables and firmware-level implants are outside a disk acquisition entirely.
Secure Enclave state	Key material, biometric templates and the hardware key hierarchy never leave the SEP.
Encrypted free space	Deleted content on the FileVault volume is unrecoverable — the per-file key died with the file (§45).
Overwritten data	Copy-on-write delays reuse but does not prevent it. Anything reallocated is gone.
Server-side data	iCloud, iMessage history not synced locally, and any cloud service state exist elsewhere and require separate legal process.
Network activity	macOS keeps no persistent connection log. Beyond a few unified-log entries and browser history, outbound traffic left no trace on this disk.
Who was at the keyboard	Nothing on the disk identifies a human. Account attribution is attribution to an account, and should always be worded that way.

■ CAUTION — STATE THE LIMITS BEFORE YOU ARE ASKED

Every limitation above will be raised by someone reviewing your work. Raising them yourself, in the methodology section, costs nothing and removes the entire line of attack. An examiner who volunteers what their evidence cannot show is far more persuasive than one who has it extracted from them.

X

AUTOMATION & CORRELATION

Running the whole of Part VII by hand is a week's work and you will make mistakes. These tools do the mechanical portion so your attention goes where it belongs — on the parts that need judgement.

60 mac_apt on Linux

61 APOLLO and pattern-of-life correlation

62 Rolling your own over a mounted image

60 mac_apt on Linux

The most complete open macOS artifact suite. Roughly forty plugins covering most of Part VII, producing SQLite, XLSX and CSV output. It runs on Linux, though the reliable input mode there is `MOUNTED` against your FUSE mount rather than reading the encrypted image directly.

```
└─╼ $ cd ~/tools/mac_apt && pip install -r requirements.txt

# see what is available before committing to a long run
└─╼ $ python3 mac_apt.py -h | sed -n '/plugins/,$p'

# full run against the mounted Data volume
└─╼ $ python3 mac_apt.py -o ~/cases/001/06-analysis/macapt -x xlsx \
    MOUNTED /mnt/data/apfs5 ALL

# or targeted, which is far faster when you know what you want
└─╼ $ python3 mac_apt.py -o ~/cases/001/06-analysis/macapt \
    MOUNTED /mnt/data/apfs5 \
    UNIFIEDLOGS FSEVENTS QUARANTINE SPOTLIGHT SAFARI CHROME \
    RECENTITEMS AUTOSTART INSTALLHISTORY USERS NETWORKING WIFI NOTIFICATIONS
```

■ CAUTION — RUN IT AGAINST THE SYSTEM VOLUME SEPARATELY

`MOUNTED` mode points at one volume. Pointed at the Data volume it will not see `/System/Library/LaunchDaemons`, because that is on the System volume (§12.3). Either run it twice, once per volume, or point it at the reconstructed unified view from §12.4. A single Data-volume run silently produces an incomplete autostart picture.

▸ TIP — AND AGAINST EVERY SNAPSHOT

The same argument as everywhere else in this manual. Script it across all mounted snapshots and diff the resulting databases; you get a per-snapshot artifact history essentially for free.

```
└─╼ $ for m in /mnt/snap*; do
      python3 ~/tools/mac_apt/mac_apt.py -o ~/cases/001/06-analysis/macapt-$(basename $m) \
        MOUNTED $m AUTOSTART QUARANTINE RECENTITEMS
    done
```

61 APOLLO and pattern-of-life correlation

APOLLO runs several hundred curated queries across the pattern-of-life databases from §28 and normalises everything onto one timeline. It is the fastest route from "I have knowledgeC.db" to "here is what the user was doing".

```
# gather the databases APOLLO wants, preserving structure
└─╼ $ mkdir -p ~/cases/001/03-extract/pol
└─╼ $ cp -a /mnt/data/apfs5/private/var/db/CoreDuet          ~/cases/001/03-extract/pol/
└─╼ $ cp -a /mnt/data/apfs5/private/var/db/powerlog          ~/cases/001/03-extract/pol/
└─╼ $ cp -a "/mnt/data/apfs5/Users/jdoe/Library/Application Support/Knowledge" \
      ~/cases/001/03-extract/pol/

└─╼ $ cd ~/tools/APOLLO
└─╼ $ python3 apollo.py -o csv -p macos ~/cases/001/03-extract/pol/
└─╼ $ head -5 apollo.csv
```

▸ TIP — MERGE IT INTO THE SUPER-TIMELINE

APOLLO's CSV drops straight into Timesketch alongside the plaso timeline and the parsed unified log. Three independent views of the same hours, on one axis, is where correlation actually happens — and it is exactly the setup §53 assumes.

62 Rolling your own

Eventually you need something no tool does. The two useful entry points are `pyfsapfs` for structural work and plain filesystem walking for artifact work.

62.1 Structural, via pyfsapfs

```
└──╼ $ python3 - <<'PY'
import pyfsapfs

c = pyfsapfs.container()
c.open('/evidence/mbp-001.dd')          # some builds want an offset-aware file object

print('volumes:', c.number_of_volumes)
for i in range(c.number_of_volumes):
    v = c.get_volume(i)
    print(f'{i}: {v.name}  encrypted={v.is_locked()}')
    if v.is_locked():
        v.set_password('TheUserPassword')
        v.unlock()
    print('   snapshots:', v.number_of_snapshots)

def walk(entry, path=''):
    """Yield every file entry with its reconstructed path."""
    for i in range(entry.number_of_sub_file_entries):
        ch = entry.get_sub_file_entry(i)
        p = f'{path}/{ch.name}'
        yield ch, p
        if ch.number_of_sub_file_entries:
            yield from walk(ch, p)
PY
```

62.2 Artifact work over the mount

For most tasks the mount is enough and far simpler. A pattern that comes up constantly — sweep a tree, parse a format, emit timeline rows:

```
└──╼ $ python3 - <<'PY'
import os, plistlib, csv, sys, datetime as dt

ROOTS = ['/mnt/data/apfs5/Library/LaunchDaemons',
         '/mnt/data/apfs5/Library/LaunchAgents']
w = csv.writer(sys.stdout)
w.writerow(['mtime','path','label','program','runatload','keepalive'])

for root in ROOTS:
    for dirpath, _, files in os.walk(root):
        for fn in files:
            if not fn.endswith('.plist'): continue
            p = os.path.join(dirpath, fn)
            try:
                with open(p,'rb') as f: d = plistlib.load(f)
            except Exception as e:
                w.writerow(['', p, f'UNPARSEABLE: {e}', '', '', '']); continue
            prog = d.get('Program') or ' '.join(d.get('ProgramArguments', []))
            w.writerow([
                dt.datetime.utcfromtimestamp(os.path.getmtime(p)).isoformat(),
                p, d.get('Label',''), prog,
                d.get('RunAtLoad',''), bool(d.get('KeepAlive'))])
PY
```

▸ TIP — EMIT TIMELINE ROWS, NOT REPORTS

Have every script you write output a row per event with an ISO-8601 UTC timestamp in the first column. Then everything you build merges with everything else, and with plaso, with one `sort`. Scripts that print prose have to be re-read; scripts that emit timeline rows compound.

XI

REPORTING

The examination is only worth what you can defend. This is short, because the discipline is simple and mostly consists of having done the recording work described in Part II.

63 Findings, manifests and defensibility

63 Findings, manifests and defensibility

63.1 The shape of a finding

Every reported finding should carry six things. If you cannot supply all six, it is a lead, not a finding.

Element	Example
Artifact	Launch agent plist, full original path, on the Data volume
Evidence	SHA-256 of the extracted file; inode number; the exhibit it belongs to
Method	Extracted via `apfs-fuse` <commit>, parsed with `plistutil` 2.3.0
Observation	What the artifact literally says — quoted, not paraphrased
Interpretation	What you conclude, and explicitly flagged as interpretation
Corroboration	The independent witnesses (§53) that agree, and any that do not

63.2 Worked example

Finding 3 — Backdated persistence mechanism

Artifact. `/Users/jdoe/Library/LaunchAgents/com.apple.softwareupdated.plist` (Data volume, inode 2100002). SHA-256 `9c1f…4b70`. Exhibit 1, extracted 2026-08-02.

Observation. A launch agent labelled `com.apple.softwareupdated` with `RunAtLoad` true, `KeepAlive` true, executing `/usr/local/bin/sysupd`. The inode records a creation time of 2019-03-14 09:12:00 UTC and a change time of 2026-07-30 02:39:11 UTC.

Interpretation. The file was created on 2026-07-30 and its timestamps subsequently altered to appear historic. The label imitates an Apple service; genuine Apple launch agents reside on the sealed System volume, not in a user's `LaunchAgents` directory.

Corroboration. (i) Inode 2100002 falls between inodes created at 02:38:44 and 02:38:51 on 2026-07-30. (ii) All four timestamps carry zero nanoseconds, indicating they were set programmatically. (iii) The file is absent from the local snapshot taken 2026-07-30 02:15 and present in the snapshot taken 03:15. (iv) FSEvents records a creation event for that path with no corresponding event in 2019. (v) No `settimeofday` event appears in the unified log for the relevant period, excluding a system clock change as an explanation.

63.3 Manifests and reproducibility

```
# hash every exhibit and every derived product
└─╼ $ hashdeep -c sha256 -r ~/cases/001/07-report/exhibits > ~/cases/001/07-report/exhibits.manifest

# record the environment that produced the findings
└─╼ $ { uname -a; lsb_release -a; fls -V; fsapfsinfo -V;
   git -C ~/tools/apfs-fuse rev-parse --short HEAD;
   pip freeze; } > ~/cases/001/07-report/toolchain.txt
```

■ CAUTION — SEPARATE OBSERVATION FROM INTERPRETATION, ALWAYS

"The plist executes `/usr/local/bin/sysupd` at load" is an observation and is not open to argument. "The machine was compromised" is an interpretation and is. Keeping them in separate, labelled sentences means that if an interpretation is later challenged or overturned, your observations — and the rest of the report — survive intact. Reports that blend the two lose everything when one inference fails.

A–G

APPENDICES

The parts you will actually open mid-case: the install script, the path tables, the timestamp arithmetic, the queries, and the command card.

A	The full Parrot / Debian install script
B	Artifact path cheat sheet (Ventura+)
C	Timestamp conversion reference
D	SQLite query cookbook
E	Command quick-reference card
F	Tool capability matrix
G	Further reading and sources

A The full Parrot / Debian install script

Everything this manual uses, in one place. Read it before running it; it installs a lot and builds several things from source.

```
#!/usr/bin/env bash
# macOS/APFS forensics toolchain for Parrot OS / Debian
set -euo pipefail
TOOLS="$HOME/tools"; mkdir -p "$TOOLS"

### 1. base system ------------------------------------------------------
sudo apt update
sudo apt install -y \
  build-essential cmake git pkg-config autoconf automake libtool flex bison \
  dkms linux-headers-"$(uname -r)" \
  libfuse3-dev fuse3 libbz2-dev zlib1g-dev libssl-dev libattr1-dev \
  python3-dev python3-pip python3-venv \
  cargo rustc

### 2. imaging and evidence handling ----------------------------------
sudo apt install -y \
  ewf-tools libewf-dev afflib-tools \
  gddrescue dc3dd dcfldd guymager \
  hashdeep

### 3. classical forensics --------------------------------------------
sudo apt install -y \
  sleuthkit libtsk-dev autopsy \
  testdisk foremost scalpel bulk-extractor binwalk magicrescue \
  sqlite3 sqlitebrowser libplist-utils \
  exiftool yara radare2 clamav xxd ent jq

### 4. APFS access engines --------------------------------------------
# 4a. apfs-fuse — day-to-day mounting, snapshots, compression
git clone https://github.com/sgan81/apfs-fuse.git "$TOOLS/apfs-fuse"
cd "$TOOLS/apfs-fuse" && git submodule update --init
mkdir -p build && cd build
cmake .. -DUSE_FUSE3=ON && make -j"$(nproc)" && sudo make install && sudo ldconfig

# 4b. libyal libfsapfs — metadata depth, encryption, plaso backend
git clone https://github.com/libyal/libfsapfs.git "$TOOLS/libfsapfs"
cd "$TOOLS/libfsapfs"
./synclibs.sh && ./autogen.sh
./configure --enable-python --with-libfuse
make -j"$(nproc)" && sudo make install && sudo ldconfig

# 4c. kernel module + apfsprogs — speed and apfsck
git clone https://github.com/linux-apfs/linux-apfs-oot.git "$TOOLS/linux-apfs-oot"
cd "$TOOLS/linux-apfs-oot" && make -j"$(nproc)" && sudo make install
sudo depmod -a && sudo modprobe apfs
git clone https://github.com/linux-apfs/apfsprogs.git "$TOOLS/apfsprogs"
for p in apfsck apfs-snap apfs-label; do
  ( cd "$TOOLS/apfsprogs/$p" && make -j"$(nproc)" && sudo make install )
done

### 5. python environment ---------------------------------------------
python3 -m venv "$TOOLS/venv-dfir"
source "$TOOLS/venv-dfir/bin/activate"
pip install --upgrade pip wheel
pip install \
  plaso pytsk3 pyewf construct \
  lief flare-capa capstone \
  biplist ccl-bplist nska-deserialize ds-store \
  pandas python-dateutil xlsxwriter

### 6. macOS artifact parsers -----------------------------------------
cd "$TOOLS"
for r in ydkhatri/mac_apt \
         mandiant/macos-UnifiedLogs \
```

```
          ydkhatri/UnifiedLogReader \
          dlcowen/FSEventsParser \
          ydkhatri/spotlight_parser \
          mac4n6/APOLLO \
          n0fate/chainbreaker \
          Yara-Rules/rules ; do
  git clone --depth 1 "https://github.com/$r.git" || true
done
pip install -r "$TOOLS/mac_apt/requirements.txt"

# the Mandiant unified-log parser is Rust
( cd "$TOOLS/macos-UnifiedLogs" && cargo build --release --examples )

# Apple code-signature verification on Linux
cargo install apple-codesign

### 7. SQLite recovery ------------------------------------------------
cd "$TOOLS"
git clone https://github.com/inflex/undark.git && ( cd undark && make )
git clone https://github.com/n0fate/walitean.git || true

### 8. record what was installed -------------------------------------
{ date -u +%FT%TZ; uname -a
  for d in "$TOOLS"/*/; do
    [ -d "$d/.git" ] && echo "$(basename "$d") $(git -C "$d" rev-parse --short HEAD)"
  done
  pip freeze; } > "$TOOLS/toolchain-manifest.txt"
echo "done — manifest at $TOOLS/toolchain-manifest.txt"
```

● **NOTE — OPTIONAL EXTRAS**

Not in the script because they are large or situational: **Timesketch** (Docker, for timeline correlation — §20), **Ghidra** (deep reverse engineering — §57), **NSRL hash sets** (large download, and see §46.3 for why the sealed volume may serve you better), and **Volatility 3** (only useful if you ever obtain a memory image — §59).

B Artifact path cheat sheet 13+

All paths relative to the mounted volume. **D** = Data volume, **S** = System volume, **P** = Preboot.

Vol	Path	Artifact
D	private/var/db/diagnostics/	Unified log .tracev3
D	private/var/db/uuidtext/	Unified log string tables — **take both**
D	.fseventsd/	FSEvents (also on every other volume)
D	.Spotlight-V100/Store-V2/<UUID>/store.db	Spotlight metadata
D	private/var/log/install.log*	Install and update history
D	private/var/audit/	OpenBSM audit trail (if enabled)
D	private/var/db/CoreDuet/Knowledge/knowledgeC.db	Pattern of life, system
D	private/var/db/CoreDuet/People/interactionC.db	Contact interactions
D	private/var/db/powerlog/Library/BatteryLife/CurrentPowerlog.PLSQL	Per-process activity
D	private/var/db/SystemPolicyConfiguration/ExecPolicy	**First-execution records**
D	private/var/db/SystemPolicyConfiguration/KextPolicy	Approved kernel extensions
D	private/var/db/com.apple.backgroundtaskmanagement/BackgroundItems-v*.btm	**BTM login items**
D	private/var/db/com.apple.xpc.launchd/disabled*.plist	launchd enable/disable state
D	private/var/db/ConfigurationProfiles/	Configuration profiles, MDM
D	private/var/db/dslocal/nodes/Default/users/*.plist	Local accounts + password hashes
D	private/var/at/tabs/*	User crontabs
D	private/etc/{sudoers,pam.d,ssh,hosts}	Unix configuration
D	Library/Launch{Daemons,Agents}/	System-wide persistence
D	Library/Application Support/com.apple.TCC/TCC.db	System privacy grants
D	Library/Keychains/System.keychain	System credentials
D	private/var/db/SystemKey	Unlocks the system keychain
D	Library/SystemExtensions/db.plist	Installed system extensions
D	Library/Security/SecurityAgentPlugins/	Login-window code
D	Library/Audio/Plug-Ins/HAL/	Audio plugin persistence
D	usr/local/{bin,sbin}/	Writable despite the path
D	Users/<u>/Library/LaunchAgents/	Per-user persistence
D	Users/<u>/Library/Preferences/com.apple.LaunchServices.QuarantineEventsV2	Download history
D	Users/<u>/Library/Application Support/com.apple.sharedfilelist/*.sfl2	Recent items
D	Users/<u>/Library/Safari/History.db	Safari browsing
D	Users/<u>/Library/Messages/chat.db	iMessage / SMS
D	Users/<u>/Library/Keychains/login.keychain-db	User credentials
D	Users/<u>/.zsh_history	Shell history
S	System/Library/Launch{Daemons,Agents}/	Apple's jobs — sealed
S	System/Library/CoreServices/SystemVersion.plist	OS version and build
S	usr/share/firmlinks	The firmlink table
P	<VolUUID>/LocalPolicy/*.img4	**Boot security posture**
P	Cryptexes/	OS/App cryptexes, RSR state

C Timestamp conversion reference

Format	Epoch / unit	Convert to Unix
APFS inode times	1970-01-01, nanoseconds	value / 1e9
Mac absolute / Cocoa	2001-01-01, seconds	value + 978307200
Core Data	2001-01-01, seconds (often float)	value + 978307200
WebKit / Chrome	1601-01-01, microseconds	value / 1e6 - 11644473600
Firefox	1970-01-01, microseconds	value / 1e6
HFS+ legacy	1904-01-01, seconds	value - 2082844800
APFS xattr timespec	1970-01-01, sec + nsec pair	first 8 bytes LE = seconds
Quarantine xattr	2001-01-01, hex seconds	int(hex,16) + 978307200

```
# a one-liner that handles the common cases
└──╼ $ cat > ~/bin/ts <<'PY'
#!/usr/bin/env python3
import sys, datetime as dt
v = float(sys.argv[1]); kind = sys.argv[2] if len(sys.argv) > 2 else 'mac'
o = {'unix':0, 'mac':978307200, 'cocoa':978307200, 'hfs':-2082844800}
if kind == 'webkit': u = v/1e6 - 11644473600
elif kind == 'apfs': u = v/1e9
elif kind == 'firefox': u = v/1e6
else: u = v + o[kind]
print(dt.datetime.utcfromtimestamp(u).strftime('%Y-%m-%d %H:%M:%S UTC'))
PY
└──╼ $ chmod +x ~/bin/ts

└──╼ $ ts 774074400 mac
2026-07-30 02:40:00 UTC
└──╼ $ ts 13398374400000000 webkit
2025-06-11 00:00:00 UTC
```

▸ TIP — THE SANITY CHECK

Result in **1988**? You treated Mac absolute time as Unix. Result in **2058**? You added the offset twice. Result in **1601 or 1904**? You used the wrong epoch entirely. These three mistakes account for nearly every timestamp error in macOS forensics, and all three are obvious on sight.

D SQLite query cookbook

Quarantine — download history

```
SELECT datetime(LSQuarantineTimeStamp + 978307200,'unixepoch') AS ts,
       LSQuarantineAgentName   AS agent,
       LSQuarantineDataURLString   AS file_url,
       LSQuarantineOriginURLString AS referrer
FROM LSQuarantineEvent ORDER BY ts DESC;
```

knowledgeC — application focus with durations

```
SELECT datetime(ZOBJECT.ZSTARTDATE + 978307200,'unixepoch') AS start,
       datetime(ZOBJECT.ZENDDATE   + 978307200,'unixepoch') AS finish,
       ZOBJECT.ZVALUESTRING AS bundle,
       CAST(ZOBJECT.ZENDDATE - ZOBJECT.ZSTARTDATE AS INT) AS secs
FROM ZOBJECT WHERE ZSTREAMNAME = '/app/inFocus'
ORDER BY start DESC;
```

TCC — privacy grants, most recent first

```
SELECT datetime(last_modified,'unixepoch') AS modified,
       service, client, client_type, auth_value, auth_reason
FROM access ORDER BY last_modified DESC;
```

ExecPolicy — first execution of each binary

```
SELECT datetime(timestamp,'unixepoch') AS first_seen,
       responsible_path, file_identifier
FROM policy_scan_cache ORDER BY timestamp DESC;
```

Safari — history with visit counts

```
SELECT datetime(v.visit_time + 978307200,'unixepoch') AS ts,
       i.url, i.visit_count, v.title
FROM history_visits v JOIN history_items i ON i.id = v.history_item
ORDER BY v.visit_time DESC;
```

Chrome — history (WebKit epoch)

```
SELECT datetime(last_visit_time/1000000 - 11644473600,'unixepoch') AS ts,
       url, title, visit_count
FROM urls ORDER BY last_visit_time DESC;
```

Messages — conversations with handles

```
SELECT datetime(m.date/1000000000 + 978307200,'unixepoch') AS ts,
       h.id AS contact, m.is_from_me, m.text
FROM message m LEFT JOIN handle h ON h.ROWID = m.handle_id
ORDER BY m.date DESC LIMIT 200;
```

> **● NOTE — chat.db USES NANOSECONDS SINCE 2001**
>
> Note the `/1000000000` in that last query. Messages switched from seconds to nanoseconds in the Mac-absolute epoch around macOS 10.13. If your dates come out in 2001, the column is in nanoseconds and you divided by nothing; if they come out absurdly far in the future, it is in seconds and you divided anyway. Test on a message whose date you can verify.

Any table — find backdated rows

```
-- rowids are monotonic; a low timestamp at a high rowid was inserted late
SELECT ROWID, datetime(<timecol> + 978307200,'unixepoch') AS ts
FROM <table> ORDER BY ROWID;
```

E Command quick-reference card

Setup

```
IMG=/evidence/mbp-001.dd
OFF=1024040          # sectors
BOFF=$((OFF*512))    # bytes
LOOP=$(sudo losetup -r -P -f --show $IMG)
```

Discover

```
mmls $IMG
pstat -o $OFF $IMG
fsapfsinfo -o $BOFF $IMG
fsapfsinfo -o $BOFF -p 'PW' -f 5 -s $IMG   #
snapshots
```

Mount

```
fsapfsmount -o $BOFF -p 'PW' $IMG /mnt/apfs
apfs-fuse -v 4 -p 'PW' -o ro ${LOOP}p2 /mnt/data
apfs-fuse -v 4 -p 'PW' -s SNAPNAME -o ro ${LOOP}p2
/mnt/snap
fusermount3 -u /mnt/data
```

Write-block

```
sudo blockdev --setro $LOOP
truncate -s 64G /var/tmp/cow.img
COW=$(sudo losetup -f --show /var/tmp/cow.img)
SZ=$(sudo blockdev --getsz $LOOP)
echo "0 $SZ snapshot $LOOP $COW P 8" | \
  sudo dmsetup create evid-cow
```

Sleuth Kit

```
fsstat -o $OFF -B $DAT $IMG
fls   -o $OFF -B $DAT -r -p $IMG
fls   -o $OFF -B $DAT -r -p -d $IMG   # deleted
istat -o $OFF -B $DAT $IMG <inode>
icat  -o $OFF -B $DAT $IMG <inode> > out
tsk_recover -o $OFF -B $DAT -e $IMG out/
```

Timeline

```
fls -o $OFF -B $DAT -r -m / $IMG > body.txt
mactime -b body.txt -d -z UTC > tl.csv
log2timeline.py --storage_file p.db \
  --credential password:'PW' $IMG
psort.py -o dynamic -w super.csv p.db
```

Artifacts

```
unifiedlog_parser -i logs/ -o ul.csv
python3 FSEParser_V4.1.py -s .fseventsd -o out/
python3 spotlight_parser.py store.db out/
python3 mac_apt.py -o out MOUNTED /mnt/data ALL
python3 apollo.py -o csv -p macos dbs/
```

xattrs

```
getfattr -R -d -m '.*' -e base64 /mnt/data
getfattr -n com.apple.quarantine -d FILE
getfattr -n com.apple.macl -e hex FILE
```

F Tool capability matrix

Task	First choice	Fallback / notes
Mount encrypted APFS	`fsapfsmount -p`	`apfs-fuse -p`; kernel module cannot
Mount a snapshot	`apfs-fuse -s`	`fsapfsinfo` to enumerate first
Container metadata	`fsapfsinfo`	`pstat` for TSK coordinates
Inode-level work	TSK `istat`/`icat`	Unencrypted volumes only, in practice
Filesystem timeline	`fls -m` + `mactime`	plaso for the merged super-timeline
Structural integrity	`apfsck`	Against the COW overlay, never the image
Unified log	Mandiant `macos-UnifiedLogs`	`UnifiedLogReader` for stubborn files
Bulk artifacts	`mac_apt`	Run per volume and per snapshot
Pattern of life	APOLLO	Hand-written SQL for anything it lacks
Deleted DB rows	`sqlite3 .recover`	`undark`, `walitean` for WAL
NSKeyedArchiver	`nska_deserialize`	`ccl_bplist`
Code signatures	`rcodesign`	`lief` for structure only
Carving	`bulk_extractor`	`photorec`; expect poor results (§19)
Keychain	`chainbreaker`	System keychain needs no password

G Further reading and sources

Primary specification

- **Apple File System Reference** — Apple's own on-disk format document. The authority for every structure named in Part IV. Terse, complete, occasionally out of date on newer features.

Tooling

- `libyal/libfsapfs` — the documentation directory contains the most thorough independent write-up of APFS internals available, including encryption.
- `sgan81/apfs-fuse`, `linux-apfs/linux-apfs-oot`, `linux-apfs/apfsprogs`
- `ydkhatri/mac_apt` — and its wiki, which documents artifact locations per macOS version.
- `mandiant/macos-UnifiedLogs` — plus Mandiant's write-ups on the tracev3 format.
- `mac4n6/APOLLO` — Sarah Edwards' module set; the modules themselves are a reference on pattern-of-life databases.

Practice

- Sarah Edwards, *Mac and iOS Forensic Analysis* and the mac4n6 blog — the standard reference for pattern-of-life work.
- Objective-See's blog and tool set — the best public catalogue of real macOS persistence and malware behaviour. The `Malware` repository is a legitimate corpus for testing your YARA rules and triage workflow.
- Patrick Wardle, *The Art of Mac Malware* — freely available, and the best explanation of why the persistence surfaces in Part VIII are the ones that matter.

> **■ CAUTION — VERIFY EVERYTHING AGAINST YOUR OWN IMAGE**
>
> This manual, like every macOS forensics reference, describes a moving target. Paths change between point releases, tools change behaviour between commits, and Apple documents its filesystem when it feels like it. Where a finding matters, confirm the artifact exists where this document says it does, on your build number, and record the version you observed. Treat every path here as a strong hint rather than a guarantee — that is the correct posture for the whole field, not a weakness of this document.
